

Post-Quantum Signature Schedules for Streaming LLM Inference: A Measurement Study with PQ-Shield

Nivitha K

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India
x@vit.com

Manav Prakash

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India
x@vit.com

Shashwat Kumar

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India
x@vit.com

Yatharth Srivastava

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India
x@vit.com

Abstract—Large language models stream their responses token by token, so a response cannot be signed as a whole before its first byte is sent. With post-quantum signatures this matters: an ML-DSA-65 signature is 3,309 B against ≈ 71 B for ECDSA. We measure the resulting trade-off with PQ-Shield, an application-layer emulation of protected inference transactions (not a TLS implementation), on a real Llama-3.2-3B backend. Treating per-chunk and hash-chained signing as the endpoints of a hash chain checkpointed every k chunks, we measure signature bytes against exposure, the tokens a client displays before a verified signature covers them: from 135,669 B and 5 tokens at $k=1$ to 3,309 B and the whole 200-token response at $k=\infty$, with byte counts that match an analytic model exactly. Per-signature time inside a stream reaches up to $18\times$ a back-to-back microbenchmark, because the processor slows between chunks; we bound it rather than predict it. Binding role, session, and position into every signed message, a signed end-of-stream record, and client enforcement of the requested checkpoint schedule detected every one of 135,000 trials of static and adaptive attacks across six configurations (60 distinct streams per attack and cell), with no false rejections in 750 genuine streams; without the schedule check, stripping checkpoints went undetected in all 9,000 trials. Detection, however, arrives only with a signature, and k bounds how much unverified text was shown by then. Against an unauthenticated handshake, key substitution succeeded in all 1,800 trials regardless of post-quantum primitives; a transcript signature under a pinned identity key stopped all 3,600 attempts at no resolvable latency cost. Key-establishment results, measured with a matched no-crypto protocol control on a single host, confirm prior TLS studies: at 10 connections, ML-KEM configurations add 1.3–5.4 ms of cryptographic overhead per 92 ms transaction and are equivalent to X25519 within $\pm 5\%$. Our ML-KEM-768 and ML-DSA-65 bindings reproduce all 130 tested NIST ACVP vectors, including ML-DSA key generation and signing.

Index Terms—post-quantum cryptography, ML-DSA, ML-KEM, stream authentication, LLM streaming, hybrid key exchange, harvest-now-decrypt-later, inference APIs, measurement

I. Introduction

AI inference endpoints—hosted by commercial providers, on managed cloud platforms, or as custom FastAPI/Flask services—carry model inputs and outputs that can be sensitive: diagnostic features, financial records, and increasingly the full text of conversational and agentic sessions. Inference endpoints are already a recognised attack surface for interception, tampering, and privacy leakage [10], [11]. Their transport protection rests on RSA or elliptic-curve key exchange and signatures, all of which Shor’s algorithm breaks in polynomial time on a sufficiently large fault-tolerant quantum computer [21]. Such a machine does not yet exist, but the harvest-now-decrypt-later (HN DL) threat means an adversary can record encrypted traffic today and decrypt it once one does; data whose confidentiality must outlive the arrival of a CRQC is already exposed [13], [22].

NIST finalized its first post-quantum standards in August 2024: ML-KEM (FIPS 203) for key encapsulation [16] and ML-DSA (FIPS 204) for signatures [17]. Protocol-level migration is under way—the X25519MLKEM768 hybrid group is already negotiated by major browsers and content-delivery networks [20]—and post-quantum TLS has been benchmarked extensively [6], [14], [24], [25]. For key establishment, that literature has largely settled the cost question: post-quantum key exchange adds little computation, and its larger messages matter mainly on slow or lossy links. Less is known about signatures on responses that are streamed. A large language model (LLM) returns its answer token by token, so the response cannot be signed as a whole before the first byte is sent. Signing every chunk is then the obvious design, and with a post-quantum signature it is an expensive one: an ML-

DSA-65 signature is 3,309B against ≈ 71 B for ECDSA P-256. Amortizing one signature over many chunks is a classical idea [26]–[28], but it trades bytes for a window in which the client shows text no signature yet covers. How large that window is, what it costs, and how an adversary can exploit it have not, to our knowledge, been measured for post-quantum signatures on LLM streams.

This paper is a measurement study of that trade-off, carried out with PQ-Shield, an application-layer emulation of protected inference transactions (not a TLS implementation). Its contributions are:

- 1) The signature schedule as the post-quantum cost of streaming. On a real Llama-3.2-3B backend, we treat per-chunk and hash-chained signing as the endpoints of one design space—a hash chain checkpointed every k chunks—define exposure (the tokens a client displays before a verified signature covers them), and measure signature bytes, exposure, and verification lag for $k \in \{1, 2, 5, 10, 20, \infty\}$. Signature counts and bytes match an analytic model exactly; we also explain why per-signature time inside a stream exceeds microbenchmarks.
- 2) Stream integrity under static and adaptive attack. We show that naive per-chunk signing admits undetected truncation, fix it by binding role, session, and position into every signed message and ending each stream with a signed length record, and evaluate drop, reorder, duplication, replay, truncation, whole-session replay, and five attacks that target checkpoints (500 trials per cell, with confidence bounds and a false-rejection baseline). One adaptive attack—stripping checkpoints—defeats a client that does not enforce the checkpoint schedule it requested; we close it.
- 3) The adversary an unauthenticated handshake invites. A key-substitution attack against a live server succeeds against the unauthenticated handshake every time and fails every time once the client checks a transcript signature under a pinned identity key, whose cost we measure.
- 4) Key establishment, as confirmation. Seven configurations—including an X25519 baseline, the deployed X25519MLKEM768 hybrid, its combination with ML-DSA-65, and a matched no-crypto protocol control—under concurrency (10–1,000 connections), four payload shapes, byte-level network emulation, and session reuse, analyzed with repetition-level statistics. These results confirm, in an inference setting, what the post-quantum TLS literature found for generic traffic.
- 5) Ground truth. Our ML-KEM-768 and ML-DSA-65 bindings reproduce all 130 tested NIST ACVP known-answer vectors, including ML-DSA-65 key generation and signing byte for byte.

II. Background and Related Work

A. Post-Quantum Primitives

ML-KEM-768 (FIPS 203) is a module-lattice key-encapsulation mechanism at NIST security category 3, with public key, secret key, and ciphertext sizes of 1,184, 2,400, and 1,088 bytes and a 32-byte shared secret. ML-DSA-65 (FIPS 204) is a module-lattice signature scheme at the same category, with public key, secret key, and signature sizes of 1,952, 4,032, and 3,309 bytes. Several publications that post-date FIPS 204 still quote pre-standardization CRYSTALS-Dilithium3 values (a 3,293-byte signature and 4,000-byte secret key); all values in this paper are confirmed against FIPS 204 as finalized (Section VII). The X25519MLKEM768 hybrid [20] concatenates an ML-KEM-768 share with a 32-byte X25519 share and derives the session key from both shared secrets, so it stays secure unless both algorithms are broken.

B. Related Work

Table I synthesizes fifteen studies spanning heterogeneous-hardware PQC benchmarking, industry deployment, layered TLS 1.3 evaluation, migration surveys, 5G/6G integration, ML threat modeling, and HNDL economics. Among the studies we reviewed, we found none that measures post-quantum signatures on streamed model responses, and none that evaluates migration with AI inference payloads under load together with adversarial experiments; we do not claim this survey is exhaustive. Three further lines of work bear directly on our design.

Post-quantum TLS under network conditions: Paquin et al. [24] benchmarked post-quantum key exchange and signatures inside TLS 1.3 over emulated links (Linux netem), showing that on lossy or low-bandwidth paths message size, not computation, dominates handshake latency. Sikeridis et al. [25] reached a similar conclusion for post-quantum authentication, and Montenegro et al. [6] and Gómez-Cambronero et al. [14] extend the analysis to layered TLS and application-level load. These works measure the TLS handshake with generic payloads. PQ-Shield instead measures an application-layer protocol around AI inference payloads, including streaming responses, and adds controlled adversarial experiments. Our network emulation (Section V-H) is deliberately coarser than theirs—it shapes the byte stream rather than packets—and we use it to bound, not replace, their packet-level results.

Stream authentication: Signing a stream that is not available in full when transmission starts is a classical problem. Gennaro and Rohatgi [26] chain per-block hashes so that one signature authenticates an entire stream; Wong and Lam [27] amortize one signature over a flow with hash trees; TESLA and EMSS [28] trade immediate verification for low per-packet overhead. Our hash-chain strategy is an instance of this family, not a new construction, and inherits the same trade-off: integrity is

TABLE I
Synthesis of related work and the gap each leaves open

Study	Focus	Not covered (relative to this study)
Abbasi et al. [1]	Heterogeneous-hardware PQC benchmark	No AI workload or network stack
Demir et al. [2]	Industry TLS/telecom PQC deployment	No application-layer API metrics
Egbuagha & Ikwunna [3]	Protocol-level PQC readiness review	No experimental data
Pote & Bansode [4]	Micro-benchmark of core PQC operations	No web protocol context
Shaw [5]	Python X25519+ML-KEM hybrid library	No adversarial threat model
Montenegro et al. [6]	Post-quantum TLS 1.3 evaluation framework	Restricted to generic TLS traffic
Chhetri et al. [7]	Quantum-safe migration survey	Qualitative; no performance data
Faval et al. [8]	PQC in 5G core networks	Telecom-specific profiles
Kudaloor & Aijaz [9]	PQC for 6G wireless nodes	No REST/JSON API pipeline
Song et al. [10]	Privacy risks of robust ML models	Pre-dates practical PQC standards
Tidjon & Khomh [11]	Threat assessment for ML systems	Ignores transport-layer crypto
Ahmed et al. [12]	PQC support in crypto libraries	No end-to-end measurement
Blanco-Romero et al. [13]	Economic feasibility of HNDL	No application-layer context
Gómez-Cambronero et al. [14]	Layered TLS 1.3 handshake performance	Fixed key exchange; no signature sweep
Schemitt et al. [15]	PQC signatures on blockchains	Domain-restricted to blockchain

confirmed only when a signature arrives. What is new is the setting. LLM token streams have a small, latency-critical first chunk and, with post-quantum signatures, a per-signature cost large enough that the choice of amortization decides whether streaming is affordable at all; we make that choice explicit and measurable through checkpointing (Section V-F).

Side channels in encrypted LLM streams: Weiss et al. [29] recovered the content of encrypted AI-assistant responses from the sizes of streamed packets (the token-length side channel). The attack is orthogonal to the key exchange and applies to every protected configuration here, but it sharpens one qualification: per-chunk and hash-chained signing emit one record per chunk and therefore expose chunk count and timing, which only buffer-and-sign hides.

III. Threat Model

A. Harvest-Now-Decrypt-Later (HNDL)

A passive adversary records every cryptographic artifact on a protected connection—the key-establishment blob, every AES-256-GCM (nonce, ciphertext) pair, and any signatures—and stores them until a CRQC becomes available. We assume AES-256-GCM itself remains secure (Grover’s algorithm only halves its effective key length), so what differs between configurations is whether the recorded handshake lets the adversary recover the session key. Under Shor’s algorithm this holds for RSA-OAEP and X25519. For ML-KEM-768 and X25519MLKEM768 it does not under currently known quantum attacks, assuming a correct implementation. This is a statement about the modeled key-recovery attack on the recorded handshake, not a proof that the protocol resists every future attack. The quantity we measure is the byte volume that becomes decryptable, not its information content; note also that response lengths already leak through packet sizes without any quantum computer [29], so the incremental harm of HNDL is the plaintext content itself.

B. Active Man-in-the-Middle (MITM)

We consider two active adversaries. The first tampers with protected records in transit—it flips a bit in the AEAD ciphertext or in the signature—without touching the handshake; this tests whether, where, and how quickly the client’s checks reject modified data. The second is the adversary an unauthenticated handshake invites: it replaces the server’s key shares and signing key with its own, terminates both legs, reads the request, and re-signs a response of its choosing. No signature or AEAD check can stop this, because the client verifies under whatever key it was given. By default PQ-Shield’s handshake is unauthenticated, like an anonymous key exchange; we measure the key-substitution attack against it and against an authenticated variant in which the server signs the handshake transcript with a long-term identity key the client has pinned (Section IV), playing the role of a certificate and CertificateVerify in TLS 1.3. All latency experiments use the unauthenticated handshake unless stated otherwise.

C. Streaming-Specific Threats

Two threats appear only when one connection streams many chunks. The first is a sequence-integrity attack, in which an on-path adversary rearranges validly protected records without forging any: it may drop a chunk, reorder two chunks, duplicate a chunk, substitute a validly signed chunk recorded from another session (cross-session replay), or truncate the stream by cutting it off before its last chunks and its terminal record (forged termination). Each chunk’s AEAD tag and, under naive per-chunk signing, its signature remain valid, so detection requires binding every signed message to its role, its session, and its position, and authenticating the end of the stream. Our protocol binds a 3-byte role label, a 32-byte session context derived from the handshake identifier (one stream per handshake, so it also identifies the stream), and the chunk index into every signed message, and ends every stream with a signed record that commits to the number of

TABLE II
Configurations. ECDSA is over P-256; “PQ”: key exchange resists a CRQC. All protected configurations share AES-256-GCM with an HKDF-SHA256 session key.

Configuration	Key establishment	Signature	PQ
Control	none	none	–
A: Classical-RSA	RSA-2048-OAEP (new key per handshake)	ECDSA	no
A’: Classical-ECDHE	ephemeral X25519	ECDSA	no
B: Hybrid	ML-KEM-768	ECDSA	yes
B’: Hybrid-KEX	X25519MLKEM768	ECDSA	yes
C: Full PQC	ML-KEM-768	ML-DSA-65	yes
C’: Hybrid-KEX-PQ	X25519MLKEM768	ML-DSA-65	yes

chunks (Section V-B). We measure how early each strategy detects each attack.

The second is HNDL exposure that compounds with session length. A streamed session reuses one session key for every chunk, so a passive adversary who records the handshake captures the whole session at once: the longer the stream, the more ciphertext becomes retroactively readable the moment the key exchange is broken. We quantify this by relating the number of generated tokens to the volume a single future key recovery would expose. None of the evaluated configurations rotates keys within a session; mid-stream rekeying is left to future work.

IV. System Design

PQ-Shield is an application-layer emulation of protected inference transactions, not an implementation of TLS: it reproduces the cryptographic work of key establishment, payload encryption, and response signing around an inference call, without certificate authentication or TLS framing. Separate processes wrap the same FastAPI inference server (Fig. 1): an unprotected control and six protected configurations (Table II) sharing a common ServerCryptoConfig/ClientCryptoConfig contract, so that only the asymmetric key-establishment and signature primitives differ. All protected configurations use the same AES-256-GCM layer keyed by HKDF-SHA256 over the raw shared secret. Every protected transaction follows the same five steps—handshake, session-key establishment, encrypt-and-send, decrypt, verify (Fig. 2)—with a fresh ephemeral key pair per transaction by default.

Configuration A’ is an X25519-based classical baseline representative of the key-establishment primitive used in modern TLS deployments: TLS 1.3 removed RSA key transport [19], and ephemeral X25519 is the classical half of the X25519MLKEM768 group now negotiated on the web [20]. A’ and B have identical signature code, so A’→B isolates exactly one change, X25519 to ML-KEM-768. Configuration A uses RSA-2048-OAEP key transport with per-transaction key generation; no production system generates RSA keys per handshake, so we include A as a legacy worst-case baseline that exposes the cost of RSA key generation under load, not as the classical baseline.

Configuration B pairs a post-quantum key exchange (ML-KEM-768 alone) with a classical signature. Configuration B’ instead combines X25519 and ML-KEM-768 within the key exchange, following [20]: the server’s share is the ML-KEM encapsulation key followed by an X25519 public key (1,216 B), the client’s is the ML-KEM ciphertext followed by its X25519 public key (1,120 B), and the session key is derived from both shared secrets concatenated. With identical signature code, A’→B’ is the migration step deployed on the web today, and B→B’ the price of hedging against an ML-KEM break. Configuration C’ combines B’s key exchange with C’s ML-DSA-65 signatures: post-quantum on both axes and hedged in the key exchange. It was added in revision and is measured in the concurrency (a separate, later run), streaming, checkpoint, threat, and authentication experiments, but not in the payload, network, and session-reuse sweeps.

Authenticated handshake (optional): In the default handshake nothing binds the key shares to the server. In the authenticated variant (enabled per server), the server holds a long-term identity key—ECDSA P-256 for configurations that sign with ECDSA, ML-DSA-65 for C and C’, so a post-quantum configuration stays post-quantum—and returns, with every handshake, a signature over SHA-256(“PQ-Shield-handshake-v1” || 0x00 || *hid* || *alg_{kex}* || *alg_{sig}* || *pk_{kex}* || *pk_{sig}*), with each field length-prefixed. The client pins the identity public key once, out of band (in our harness, from a dedicated endpoint before any measured transaction, standing in for a certificate it already trusts), and rejects any handshake whose transcript signature does not verify. We do not model a PKI or certificate chains.

Streaming experiments use a real Llama-3.2-3B-Instruct model (Q4_K_M, llama.cpp with Metal). The concurrency, payload-shape, network, session-reuse, HNDL, and MITM experiments use the non-streaming endpoint with a RandomForest classifier (or the selected payload profile). One locally hosted LLM produces ≈40 tokens per second for a single stream, so a 1,000-connection LLM sweep would measure the model’s queue rather than the cryptography.

V. Methodology

A. Environment and Process Isolation

All measurements were taken on one host (Apple M3 MacBook Air, 8 cores, 16 GB, macOS) on AC power with low-power mode disabled, on a single day. Each configuration’s server runs in its own process, is health-checked, swept, and stopped before the next configuration starts, so no two configurations share the CPU. All endpoints, including the control’s, run their request handlers in the server’s worker thread pool, so the control is not disadvantaged by blocking inference on the event loop. Every sweep records a run identifier, the host, and its parameters with each raw row, and the analysis uses each configuration’s most recent run.

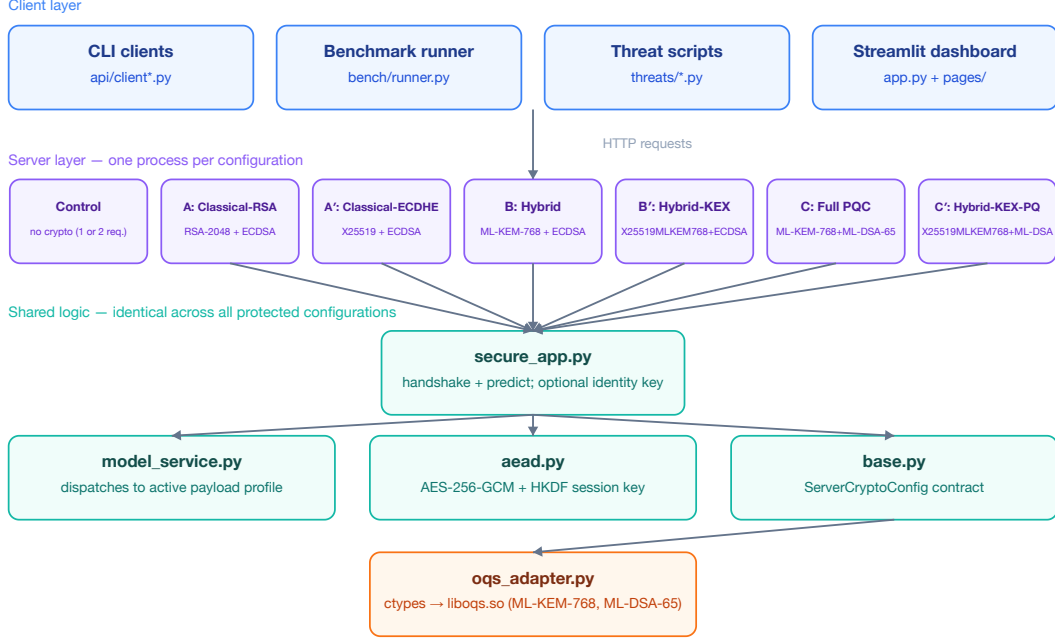


Fig. 1. PQ-Shield architecture. Separate server processes wrap the same FastAPI inference service: an unprotected control (also used, with an extra no-crypto request, as Control-2RT) and six protected configurations that share the handshake/predict endpoints, the AES-256-GCM + HKDF-SHA256 session layer, and one ServerCryptoConfig contract, so only the key-establishment and signature primitives differ. ML-KEM-768 and ML-DSA-65 come from a ctypes binding to liboqs; RSA, X25519, and ECDSA from OpenSSL via the Python cryptography package.

B. Cryptographic Binding and Protocol Construction

ML-KEM-768 and ML-DSA-65 are accessed through a purpose-built ctypes binding against a locally compiled liboqs [23] rather than the higher-level liboqs-python wrapper. The binding exposes the six C entry points the configurations use (key generation, encapsulation, and decapsulation for ML-KEM-768; key generation, signing, and verification for ML-DSA-65), plus, for validation only, context-string signing and liboqs’s hook for replacing its random-number source (Section VII), with byte lengths asserted against the FIPS 203/204 constants at every call boundary, so a build or linkage error surfaces immediately rather than propagating malformed key material. liboqs is compiled with only these two algorithms, which keeps continuous re-verification of the binding (Section VII) cheap. RSA-OAEP, X25519, ECDSA P-256, and AES-256-GCM come from OpenSSL through the Python cryptography package.

The handshake orchestration, AES-256-GCM payload encryption (keyed by HKDF-SHA256 over the raw shared secret in every configuration, including Classical-RSA’s OAEP-transported key), and FastAPI plumbing are identical code paths across configurations, so differences between them can be attributed to the asymmetric primitives. By default a fresh ephemeral key pair is generated on every transaction—the conservative worst case; Sec-

tion V-G measures the amortized case.

Stream message binding: For streaming, the same signature primitive is applied to strategy-specific messages, each beginning with a 3-byte role label and the session context $ctx = \text{SHA-256}(\text{"PQ-Shield-stream-v2"} \parallel 0x00 \parallel hid)$, where hid is the handshake identifier and i, n are 4-byte big-endian integers:

- buffer-and-sign: $\text{Sign}(\text{BUF} \parallel ctx \parallel \text{nonce} \parallel ct)$;
- per-chunk: $\text{Sign}(\text{CHK} \parallel ctx \parallel i \parallel \text{nonce} \parallel ct_i)$ for every chunk, then a terminal $\text{Sign}(\text{END} \parallel ctx \parallel n)$;
- hash-chain: $h_0 = \text{SHA-256}(\text{GEN} \parallel ctx)$,

$$h_i = \text{SHA-256}(h_{i-1} \parallel i \parallel \text{nonce}_i \parallel ct_i), \quad (1)$$

optional checkpoints $\text{Sign}(\text{CKP} \parallel ctx \parallel i \parallel h_i)$, and a terminal $\text{Sign}(\text{FIN} \parallel ctx \parallel n \parallel h_n)$.

The role label separates the five message types; ctx makes a record signed for one session fail verification in any other, even if a server reused one long-term signing key (ours are per-handshake, which also prevents it); the index binds position; and the terminal records commit to the stream length. The client requires the terminal record, checks its count against the chunks it accepted, and recomputes h_i itself rather than trusting transmitted chain hashes.

C. Concurrency Benchmark

Load is generated by an asynchronous client with a bounded worker pool, so the achieved concurrency

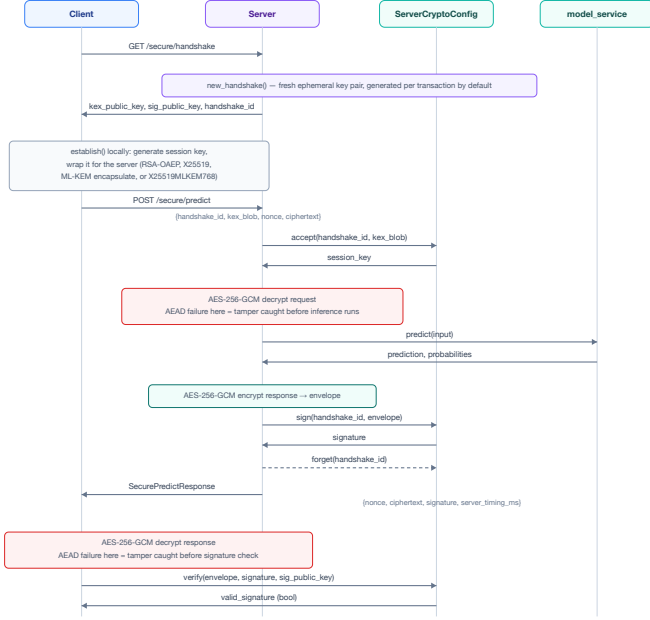


Fig. 2. One protected transaction. The client fetches fresh ephemeral keys, establishes the session key locally, and sends an encrypted request; the server decapsulates, decrypts (an AEAD failure here stops tampering before inference), infers, encrypts, and signs. The client decrypts first—an AEAD failure rejects a tampered response before any signature work—and then verifies the signature.

matches the requested level as latency varies. Besides the control (one unprotected request per transaction) we run a matched-protocol control, Control-2RT, which makes the same two HTTP requests as a protected transaction—a handshake request and the inference request—with no cryptographic work. Protected minus Control-2RT is then the cryptographic overhead alone, and Control-2RT minus control the protocol overhead of the extra round trip. Each cell (configuration $\times$ concurrency $\times$ repetition) sends ten requests per connection (with a floor for low concurrency), and five repetitions are collected at 10, 100, and 1,000 connections. The first 5% of requests in each cell are discarded as warm-up. Latency is end-to-end (total_ms: handshake round trip plus protected request), the only quantity directly comparable to the control’s single request. A sampler records server CPU and resident memory every 0.25 s. Failed requests are counted, classified by stage (handshake or protected request) and cause, and excluded from latency statistics; because failure rates differ between configurations, we repeat every comparison at the levels where failures occur with each failed request counted at the 30 s client timeout, slower than any successful request (Section VI-H).

Statistical design: Requests within one repetition share a server process, its warm-up state, and its queue, so they are not independent; the repetition is our unit of analysis. We report the median, p95, failure rate, and

the range of the five per-repetition medians. Differences between configurations are differences in medians with 95% confidence intervals from a two-stage bootstrap (1,000 draws: resample repetitions, then requests within each drawn repetition). Effect sizes are Cliff’s δ . Significance uses the two-sided Mann–Whitney U test on per-repetition medians (five versus five, so the smallest attainable p is 0.008), Holm-corrected within each family of comparisons against one reference at one concurrency level. To support claims that two configurations perform the same, we test equivalence with two one-sided tests: a configuration is declared equivalent to a reference when the 90% bootstrap CI of the median difference lies within $\pm 5\%$ of the reference median. We call a difference practically significant only if its CI excludes a 5% band, not merely zero.

D. Payload-Shape Sensitivity

Four profiles are swept at 10 connections with the control and the five original protected configurations (five repetitions): `tabular_small` (a 100-tree RandomForest trained on the UCI handwritten-digits dataset, 345 B request / 93 B response), `image_cnn` (a fixed-weight NumPy convolutional network on a synthetic $32 \times 32 \times 3$ image, 4,138 B / 244 B), `embedding` (a synthetic 768-dimension vector generator, 202 B / 8,022 B), and `llm_completion` (a synthetic text generator, 163 B request). The synthetic profiles probe payload shape and size; they are not claims of representative model behaviour.

E. Streaming Signature Strategies

The streaming endpoint sends chunks as Server-Sent Events over one HTTP response; each event carries one strategy-specific record (nonce, ciphertext, and, depending on strategy, a signature and/or chain hash). The model’s blocking token generator is bridged into the server’s event loop through a worker thread so that generation does not stall other requests. Time to first token (TTFT) is measured at the client from issuing the request to receiving the first event. Each configuration $\times$ strategy $\times$ length cell (50 and 200 tokens, 5-token chunks) has ten repetitions; after each server start one discarded generation absorbs the model’s cold start, and the order of cells is shuffled with a fixed seed so that no strategy is always measured first.

Timing signatures inside a stream: Every signing and verification call records both wall-clock time and the calling thread’s CPU time. The two differ when the thread waits—for the CPU or for Python’s interpreter lock while the model’s token loop runs in another thread—so CPU time isolates the work of the operation itself. A separate diagnostic times 300 back-to-back calls and 100 calls spaced 125 ms apart (one 5-token chunk at ≈ 40 tokens/s), each with the CPU idle and while Llama generates in the same process, to relate in-stream costs to microbenchmarks (Section VI-C).

F. Checkpointed Hash Chains and Exposure

The hash-chain strategy can additionally sign the running digest h_i every k chunks. The client verifies each checkpoint signature against its own recomputed h_i , so any dropped, reordered, or altered chunk before a checkpoint makes that checkpoint fail mid-stream. For an n -chunk response this costs $\lfloor n/k \rfloor + 1$ signatures (checkpoints plus the terminal one), and a decrypted chunk waits at most k chunks for coverage. We record two exposure measures per transaction: the largest number of chunks displayed but not yet covered by a verified signature (max unverified chunks), and the longest time any chunk waited for coverage (verification lag). Per-chunk signing is the $k=1$ endpoint (exposure one chunk) and plain hash-chain the $k=\infty$ endpoint (exposure the whole response). We sweep $k \in \{1, 2, 5, 10, 20, \infty\}$ for Hybrid (71 B signatures), Full PQC, and Hybrid-KEX-PQ (3,309 B) on 200-token Llama responses in 5-token chunks (five repetitions). Because the client requested k , it knows where checkpoints must appear and treats a missing one as a failure; Section VI-D shows why this check is needed.

G. Session Resumption

To measure the amortized case, a client may set `keep_session`: the server then caches the derived session key under the handshake identifier, later requests carry no key-establishment blob and reuse it, and the last request of the session ends it. Each of 10 concurrent clients reuses one key exchange for $N \in \{1, 10, 100\}$ requests (1,000 requests per cell, five repetitions), and we report end-to-end overhead against the control.

H. Network Emulation

All other experiments run over loopback, where moving an extra 1,088-byte ML-KEM ciphertext or 3,309-byte ML-DSA signature costs almost nothing. To expose the cost of message size we place a TCP proxy between load generator and server that adds a one-way propagation delay of $\text{RTT}/2$ in each direction and a per-direction bottleneck bandwidth shared by all connections; bytes are released in order, so HTTP semantics are unchanged. We evaluate four paths—loopback, metro (10 ms RTT, 100 Mbit/s), WAN (50 ms, 20 Mbit/s), and mobile (100 ms, 5 Mbit/s)—each with the control and the five original protected configurations at 10 connections and five repetitions. The proxy shapes the byte stream above TCP; it does not model slow start, the initial congestion window, loss, or fragmentation, all of which penalize large handshakes further [24], so the measured cost is a lower bound.

I. Threat Injection

For HNDL, the server reports, behind a debug header that does not alter ordinary traffic, the exact size of each cryptographic artifact it sends (1,000 transactions

per configuration). We distinguish harvestable bytes, everything a passive adversary captures (key-establishment blob, AEAD records, signatures, chain hashes), from decryptable bytes, the encrypted application-payload records whose plaintext becomes recoverable once the session key is recovered. For a stream of n chunks,

$$D = \mathbb{K}_{\text{KEX}} \cdot \sum_{i=1}^n (|\text{nonce}_i| + |\text{ct}_i|), \quad (2)$$

where $\mathbb{K}_{\text{KEX}}$ is 1 if a CRQC can recover the session key from the recorded handshake (RSA-OAEP, X25519) and 0 otherwise (ML-KEM-768, X25519MLKEM768), $|\text{nonce}_i| = 12$ B and ct_i includes the 16-byte GCM tag; the recoverable plaintext is $D - 28n$ bytes. Signatures, public keys, and chain hashes are excluded from D : they are not secret. One session key protects the whole stream, and each 5-token chunk is one AEAD record, so D per token depends on the chunk size (framing) and on the tokenizer (plaintext). For streaming, response length is varied with the configuration and strategy held fixed, so exposure can be attributed to length alone.

For tampering, a minimal forward proxy flips one bit in either the ciphertext or the signature field of each response (100 responses per configuration and target). The client decrypts before verifying, and detection time is the duration of the rejecting step itself—the AEAD tag check or the signature verification—so the two defence layers are directly comparable.

Stream-integrity campaign: For each protected configuration and each of five cells (per-chunk; hash chain with $k = \infty, 10, 5, 2$) we capture a pool of ten real 100-token Llama streams from a live server, verify each unmodified (false rejections), and run 500 trials per attack, each on a random pool stream at a random position, through the client’s incremental verification—the same checks the streaming client makes. Whether an attack is detected is a deterministic function of the captured stream and the attack position, so trials sample streams and positions rather than repeat network events. Static attacks are drop, reorder, duplicate, replay of a chunk from another session, truncation at a random point (losing the terminal record), and replay of an entire other session. Adaptive attacks, for checkpointed chains, strip every checkpoint signature, move a checkpoint signature to the next chunk, drop the chunk just before or just after a checkpoint, and swap a checkpoint chunk with its successor. For each detected attack we record whether it was caught before the stream ended and how many chunks the client had displayed with no verified signature covering them at that moment. Trials on the same stream differ only in attack position and are not independent, so for confidence bounds we take the captured stream as the unit (60 distinct streams per pooled entry: ten per configuration) and report exact (Clopper–Pearson) 95% intervals over streams.

Key substitution: Against a live server with the authenticated variant enabled, an in-process adversary sits between the client’s messages and the server: it substitutes its own key shares and signing key (keeping the server’s transcript signature, or re-signing the transcript with its own identity key), or relays the handshake and flips a response bit. Each variant runs 300 times per configuration against a client that ignores the transcript signature and one that checks it under the pinned key, with an unattacked baseline; we record acceptance, whether the adversary read the request, and whether a forged prediction was accepted.

J. Ground-Truth Validation

Two independent validations correspond to the two ways this paper’s results could be wrong. First, the primitive bindings are checked against official NIST Automated Cryptographic Validation Protocol (ACVP) known-answer tests [18] by re-deriving each vector’s output from its input and comparing byte for byte. *liboqs* exposes derandomized ML-KEM functions but no derandomized ML-DSA key generation or signing; however, every random byte its ML-DSA code consumes comes from one replaceable source. FIPS 204 key generation draws exactly the 32-byte seed ξ and signing exactly the 32-byte *rnd* (zero for deterministic signing), so we substitute a source that returns the vector’s bytes and check that *liboqs* drew exactly those bytes and no more. This exercises the same entry points the configurations call. Second, because no external dataset exists for streaming signature overhead, the streaming harness is checked against an analytic model: signature count and bytes follow deterministically from the number of chunks and the strategy (one signature for buffer-and-sign and plain hash-chain, one per chunk for per-chunk, $\lfloor n/k \rfloor + 1$ for checkpointed hash-chain), and signing time follows from the count and the independently measured per-signature cost.

K. Reproducibility and Artifact Availability

Every sweep records its run identifier, host, and parameters with each raw row. The software environment (OS and package versions, OpenSSL and *liboqs* revisions, the SHA-256 of the model files, and the repository commit) is captured automatically at run time and listed in Table XVII. Code, configuration, result summaries, and the scripts that regenerate every figure and table are released at <https://github.com/Yuvicodes1/PQ-Shield-Benchmarking-Post-Quantum-Security-for-AI-APIs>.

VI. Results

We report the streaming results first, because they carry the paper’s main finding, and then the key-establishment results, which confirm prior TLS work in this setting.

A. Streaming Signature Strategies

With the real Llama-3.2-3B backend (Fig. 3, Table III), buffer-and-sign delays the first token to the end of generation (5.0–5.5 s for a 200-token response). Per-chunk and hash-chain signing deliver the first token in 126–139 ms, 40–43 $\times$ sooner, in every configuration. The strategies differ in signature bytes: with ML-DSA-65 (Full PQC and Hybrid-KEX-PQ) a 200-token response carries 135,669 B of signatures under per-chunk signing (40 chunk signatures plus the signed end record) versus 3,309 B under hash-chain, a 41 $\times$ difference; with ECDSA the corresponding figures are $\approx$ 2,910 B and 71 B. All 360 streams verified completely. Strategy order was shuffled within each configuration, so comparisons between strategies are unbiased; generation speed drifted over the sweep (thermal throttling of the fanless host), so we do not compare streaming times between configurations.

B. Bounded Exposure with Checkpointed Hash Chains

Fig. 4(a) and Table IV sweep the checkpoint interval k . Signature counts and bytes follow the analytic model $(\lfloor n/k \rfloor + 1) \cdot |\sigma|$ exactly in all 18 configuration–interval cells: with ML-DSA-65, 135,669 B (41 signatures) at $k=1$, 29,781 B at $k=5$, 9,927 B at $k=20$, and 3,309 B at $k=\infty$; with ECDSA, 2,912 B down to 72 B. Exposure grows as bytes fall: the client displays at most 5, 25, 100, and 200 tokens before a verified signature covers them. The corresponding longest verification lags are 0 ms at $k=1$, 0.5–0.8 s at $k=5$, 2.7–6.0 s at $k=20$, and 5.4–6.6 s at $k=\infty$, where the whole response is unverified until its last record. The checkpoint interval is therefore a dial between signature bytes and how long a user reads unverified text; Section VI-D shows what it buys against an attacker.

C. Signing Cost Inside a Stream

Signature bytes match the model exactly, but per-signature time inside a stream does not match a back-to-back microbenchmark, and in an earlier version of this paper it appeared inconsistent: the $k=1$ checkpoint cell cost 5.7 ms of signing and verification for Full PQC but 27.3 ms for Hybrid, whose ECDSA signatures are the cheaper ones, and the implied cost per signature varied six-fold across k . Table V explains why. Signed back to back with the CPU idle, ECDSA P-256 costs 20 μ s and ML-DSA-65 73 μ s; signed once every 125 ms—the spacing of 5-token chunks—they cost 343 and 519 μ s. Thread-CPU time rises with wall time in every pattern, so the extra cost is work the processor executes, not time spent waiting for the interpreter lock or for the model’s thread. Keeping the core busy during the same 125 ms gaps removes most of it (56 and 96 μ s), and confining the thread to the efficiency cores reproduces most of it (132 and 386 μ s): between chunks the core clocks down or the thread migrates to a slower core, and each signature then runs slowly. Inside real streams, the per-signature CPU cost of every streaming and checkpoint cell lies

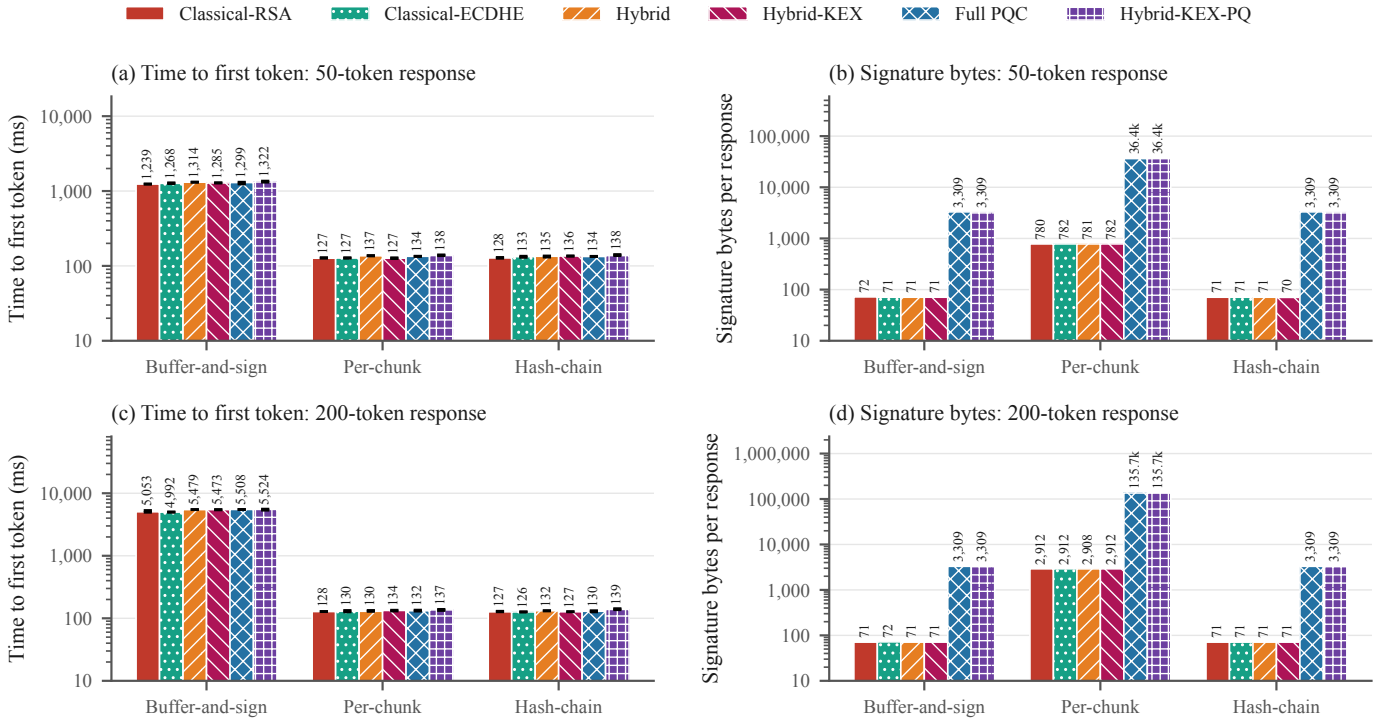


Fig. 3. Streaming signature strategies on Llama-3.2-3B (5-token chunks; median of ten repetitions, error bars interquartile range). (a, c) Time to first token: per-chunk and hash-chain deliver the first token about 40 $\times$ sooner than buffer-and-sign. (b, d) Signature bytes per response: per-chunk ML-DSA-65 signing costs 41 $\times$ the bytes of hash-chain.

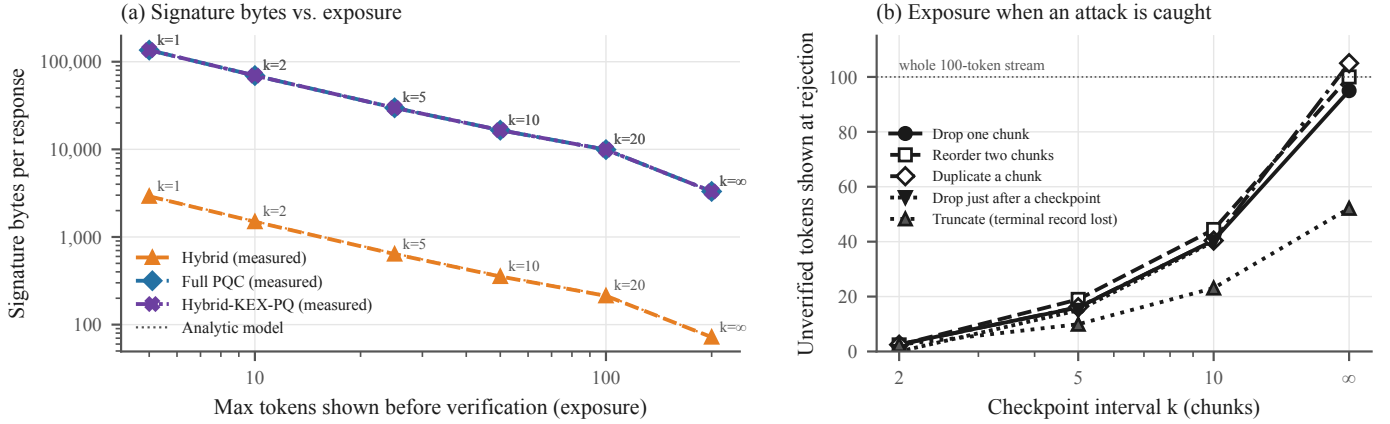


Fig. 4. Checkpointed hash chains on Llama-3.2-3B (5-token chunks). (a) Signature bytes against exposure for 200-token responses; the measurements lie on the analytic model. (b) Attack campaign on 100-token streams (pooled over six configurations, 3,000 trials per point): tokens displayed with no verified signature covering them when the client rejected the attack.

between the back-to-back and the spaced costs: 39–360 μ s for ECDSA and 92–640 μ s for ML-DSA-65, with 100% and 95% of cells within 20% of those bounds. Signing time in a stream is thus set largely by the processor’s power state, which differs from run to run and between strategies with different signing rhythms; it is bounded, not predicted, by the microbenchmark. In absolute terms it is small: at most 23 ms of signing per 200-token stream that takes 5–6 s to generate.

D. Stream Integrity Under Attack

Table VI reports the campaign: 144,000 attacked trials and 300 unmodified streams, plus the 450 streams of the streaming sweeps. No unmodified stream was rejected (0/750; 95% upper bound 0.4%). Every static attack was detected in every trial and cell (3,000/3,000 each). Because trials on one stream are correlated, the defensible statement is per stream: in each entry the attack was detected at every sampled position on all 60 streams, a 95% lower bound of 94.0% of streams. Treating the

TABLE III

Streaming signature strategies on a real Llama-3.2-3B-Instruct backend (10 repetitions per cell; median, with interquartile range for TTFT). Signing and verification times are summed over the whole stream and given as wall-clock / thread-CPU time; Section VI-C explains why per-signature cost in a stream exceeds the back-to-back microbenchmark. Generation slowed over the sweep (thermal), so compare times within a configuration, not across.

Tokens	Config.	Strategy	TTFT (ms)	Total (ms)	Sigs	Sig. bytes	Sign wall/CPU (ms)	Verify wall/CPU (ms)
50	Classical-RSA	Buffer-and-sign	1,239.0 [1,238–1,241]	1,239	1	72	0.07 / 0.07	0.08 / 0.08
		Per-chunk	126.7 [126–129]	1,236	11	780	0.43 / 0.44	0.74 / 0.74
		Hash-chain	127.7 [126–130]	1,239	1	71	0.08 / 0.08	0.07 / 0.08
	Classical-ECDHE	Buffer-and-sign	1,268.1 [1,256–1,289]	1,268	1	71	0.07 / 0.07	0.08 / 0.08
		Per-chunk	127.4 [126–129]	1,262	11	782	0.45 / 0.46	0.74 / 0.74
		Hash-chain	132.8 [130–135]	1,332	1	71	0.09 / 0.09	0.08 / 0.09
	Hybrid	Buffer-and-sign	1,313.6 [1,312–1,316]	1,315	1	71	0.33 / 0.34	0.35 / 0.35
		Per-chunk	136.5 [136–138]	1,366	11	781	2.55 / 2.59	4.57 / 4.56
		Hash-chain	134.6 [131–136]	1,350	1	71	0.34 / 0.34	0.34 / 0.34
	Hybrid-KEX	Buffer-and-sign	1,284.6 [1,280–1,290]	1,285	1	71	0.07 / 0.07	0.08 / 0.08
		Per-chunk	126.8 [125–128]	1,241	11	782	0.42 / 0.43	0.73 / 0.73
		Hash-chain	135.6 [135–136]	1,361	1	70	0.36 / 0.36	0.34 / 0.34
	Full PQC	Buffer-and-sign	1,298.6 [1,242–1,314]	1,300	1	3,309	0.46 / 0.47	0.22 / 0.22
		Per-chunk	134.5 [133–135]	1,366	11	36,399	1.15 / 1.16	0.55 / 0.55
		Hash-chain	133.7 [133–134]	1,353	1	3,309	0.09 / 0.09	0.05 / 0.05
	Hybrid-KEX-PQ	Buffer-and-sign	1,322.0 [1,312–1,358]	1,323	1	3,309	0.43 / 0.44	0.26 / 0.27
		Per-chunk	137.7 [137–140]	1,387	11	36,399	6.14 / 6.17	3.02 / 3.03
		Hash-chain	137.5 [137–142]	1,387	1	3,309	0.63 / 0.64	0.20 / 0.20
200	Classical-RSA	Buffer-and-sign	5,053.1 [4,962–5,304]	5,054	1	71	0.08 / 0.08	0.10 / 0.10
		Per-chunk	128.2 [128–129]	4,997	41	2,912	1.60 / 1.62	2.72 / 2.73
		Hash-chain	127.0 [127–130]	4,964	1	71	0.08 / 0.08	0.07 / 0.08
	Classical-ECDHE	Buffer-and-sign	4,992.2 [4,972–5,024]	4,993	1	72	0.07 / 0.07	0.09 / 0.09
		Per-chunk	129.8 [127–134]	5,396	41	2,912	6.78 / 6.80	10.00 / 9.93
		Hash-chain	126.5 [126–127]	5,025	1	71	0.08 / 0.08	0.08 / 0.08
	Hybrid	Buffer-and-sign	5,479.0 [5,457–5,495]	5,480	1	71	0.29 / 0.29	0.28 / 0.29
		Per-chunk	129.6 [129–134]	5,327	41	2,908	5.29 / 5.34	7.83 / 7.85
		Hash-chain	132.1 [131–133]	5,393	1	71	0.30 / 0.30	0.30 / 0.30
	Hybrid-KEX	Buffer-and-sign	5,472.8 [5,416–5,491]	5,474	1	71	0.34 / 0.35	0.28 / 0.29
		Per-chunk	133.6 [131–135]	5,523	41	2,912	9.77 / 9.92	15.03 / 14.90
		Hash-chain	127.2 [127–128]	5,062	1	71	0.08 / 0.08	0.08 / 0.08
	Full PQC	Buffer-and-sign	5,508.2 [5,488–5,520]	5,509	1	3,309	0.44 / 0.44	0.29 / 0.29
		Per-chunk	132.2 [130–135]	5,462	41	135,669	10.91 / 10.99	5.89 / 5.83
		Hash-chain	129.7 [127–132]	5,246	1	3,309	0.11 / 0.11	0.05 / 0.05
	Hybrid-KEX-PQ	Buffer-and-sign	5,524.3 [5,356–5,577]	5,525	1	3,309	0.32 / 0.33	0.23 / 0.23
		Per-chunk	136.5 [133–139]	5,625	41	135,669	16.25 / 16.33	7.59 / 7.64
		Hash-chain	138.9 [137–143]	5,614	1	3,309	0.47 / 0.47	0.27 / 0.27

3,000 trials as independent would give 99.88%, which the position-sampling design does not support. Per-chunk signing catches drop, reorder, duplication, and replay at the affected chunk, so no unverified text is shown; the exception is dropping the final chunk (5% of drops), which only the end record reveals. A replayed chunk or session fails AEAD decryption under this session’s key at once.

Detection is not prevention. Without checkpoints, hash-chain signing detects drop, reorder, and duplication only at the end of the stream, by which time the client has displayed the entire 100-token response unverified (19–21 chunks). Truncation is detected by every strategy only when the stream ends without its signed terminal record; with $k=\infty$ the client has by then shown on average half the response (10.4 chunks) without any signature covering it. Checkpoints bound this: the unverified text shown at rejection falls to 8–9, 3–4, and 0.5 chunks at $k = 10, 5$, and 2 (Fig. 4(b)), and every adaptive attack on checkpoints—moving a checkpoint signature, dropping

a chunk next to a checkpoint, reordering across one—is caught at the next checkpoint. One adaptive attack succeeds against a natural but incomplete client: stripping every checkpoint signature leaves a stream whose content is genuine and whose terminal signature verifies, so a client that does not check for the checkpoints it requested accepts it (0/3,000 detected in each cell) while silently losing all mid-stream assurance. Enforcing the requested schedule—a missing checkpoint is a failure—detects it at the first missing checkpoint in every trial. The results were identical across all six configurations: detection depends on where signatures fall, not on the signature scheme.

E. Key Substitution and Authenticated Handshakes

Against the default, unauthenticated handshake, key substitution succeeded in all 1,800 trials (Table VII): the adversary read every request and every forged prediction was accepted, in every configuration, post-quantum or not. Post-quantum key establishment protects recorded

TABLE IV

Checkpointed hash-chain signing on Llama-3.2-3B (200-token responses, 5-token chunks): a signature every k chunks plus the terminal one. Exposure = most tokens a client displays before a signature covers them; lag = longest wait for that coverage. Signing time summed over the stream, wall-clock / thread-CPU (median over repetitions).

Config.	k	Sigs	Bytes	Exp. (tok)	Lag (ms)	Sign (ms)
Hybrid	1	41	2,912	5	0	8.6/8.7
	2	21	1,494	10	159	4.1/4.2
	5	9	638	25	825	2.3/2.3
	10	5	354	50	1,232	1.3/1.3
	20	3	213	100	5,952	0.9/1.0
	∞	1	72	200	5,562	0.3/0.3
Full PQC	1	41	135,669	5	0	23.1/23.2
	2	21	69,489	10	135	2.1/2.1
	5	9	29,781	25	544	4.6/4.7
	10	5	16,545	50	1,613	2.0/2.0
	20	3	9,927	100	3,337	1.6/1.6
	∞	1	3,309	200	6,597	0.4/0.3
Hybrid-KEX-PQ	1	41	135,669	5	0	23.0/23.2
	2	21	69,489	10	138	10.8/10.9
	5	9	29,781	25	755	4.9/3.6
	10	5	16,545	50	1,175	0.5/0.5
	20	3	9,927	100	2,736	1.2/1.1
	∞	1	3,309	200	5,397	0.4/0.4

TABLE V

Per-call signing and verification cost (μ s, median; wall-clock / thread-CPU) by calling pattern, and per signature inside real Llama streams (CPU time; range of per-cell medians over all streaming and checkpoint cells). CPU time rises with wall time, so the extra cost after an idle gap is executed work (the core has clocked down), not waiting.

Pattern	Sign		Verify	
	ECDSA	ML-DSA	ECDSA	ML-DSA
Back-to-back, idle	20/20	73/73	49/49	36/36
Back-to-back, during generation	34/34	107/108	82/82	52/52
Back-to-back, efficiency cores	132/133	386/372	334/335	221/222
One per 125 ms, idle	343/348	519/516	392/394	244/245
One per 125 ms, core kept busy	56/57	96/97	58/58	38/38
One per 125 ms, during generation	264/268	444/448	379/381	234/201
In streams (CPU, range of cells)	39–360	92–640	66–414	48–294

traffic from a future CRQC; it does nothing against an active adversary if the client cannot tell whose keys it received. With the transcript signature checked under a pinned identity key, all 3,600 substitution attempts (with the server’s transcript signature left in place or replaced by one under the adversary’s own identity key) were rejected at the handshake, before any request was sent, and no unattacked transaction was rejected. A relayed handshake with a flipped response bit was rejected by the AEAD check in both modes. Authentication cost little (Table VIII): one extra signature per handshake (71 B with ECDSA, 3,309 B with ML-DSA-65) and a

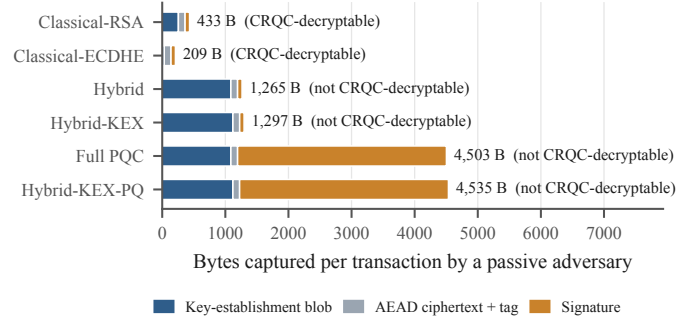


Fig. 5. Bytes a passive adversary captures per transaction, by component. Larger post-quantum captures are not decryptable by a future CRQC under the modeled attack; the smaller classical ones are.

client verification of 86 μ s (ECDSA) or 46 μ s (ML-DSA-65); end-to-end differences at 10 connections were within ± 1.7 ms with confidence intervals that include zero for every configuration.

F. Tampering With Single Responses

Every tampered response was rejected in every configuration (Table IX). A flipped ciphertext bit is caught by the AES-GCM tag in 0.007–0.009 ms (median), before the signature is examined; a flipped signature bit is caught by verification in 0.058–0.059 ms (ML-DSA-65) and 0.063–0.083 ms (ECDSA P-256). This shows that the checks work and are cheap; it does not, by itself, show resistance to a man-in-the-middle (Section VI-E).

G. Harvest-Now-Decrypt-Later Exposure

A passive adversary captures 209–433 B per classical transaction and 1,265–4,535 B per post-quantum transaction (Fig. 5). Under the modeled attack only the classical captures—RSA-OAEP and X25519 alike—become decryptable. Under streaming (Fig. 6) the decryptable volume D grows linearly with response length for both classical key exchanges, at 11.4 bytes per generated token ($R^2=0.9997$, 50–585 tokens, one model and tokenizer), of which 5.8 bytes per token are plaintext and the rest per-chunk AEAD framing. All four ML-KEM configurations have zero decryptable bytes at every length. These are byte volumes under a modeled key recovery, not measures of information content.

H. Key Establishment Under Concurrency

Table X and Fig. 7 summarize the sweep; differences are medians with 95% two-stage bootstrap CIs. At 10 connections—the setting least affected by the shared host—the matched-protocol control (Control-2RT) costs +1.7 ms [−0.9, +4.0] over the control, and the cryptographic overhead (protected minus Control-2RT) is +1.3 ms [−0.4, +3.7] for Full PQC, +2.3 [+0.3, +5.0] for Hybrid, +2.7 [+0.6, +5.2] for Classical-ECDHE, +4.2 [+1.9, +7.1] for Hybrid-KEX-PQ, +5.4 [+3.2, +7.8] for

TABLE VI

Stream-integrity campaign on real Llama streams (100 tokens, 5-token chunks), pooled over 6 configurations: 500 trials per configuration and cell, each on a random one of 10 captured streams at a random position (3,000 trials and 60 distinct streams per entry). Each entry: % detected / % detected before the stream ended / mean chunks shown with no verified signature covering them when the client rejected. Trials on the same stream differ only in attack position and are not independent, so we take the stream as the unit: 100% detected means the attack was detected at every sampled position on all 60 streams (exact 95% lower bound 94.0% of streams); 0% means it was detected on none (upper bound 6.0%). Treating trials as independent would give 99.88%, which the design does not support. Benign: unmodified streams rejected (false rejections).

Attack	Per-chunk	Chain $k=\infty$	$k=10$	$k=5$	$k=2$
Benign (false rejections)	0/60 (<4.9%)	0/60 (<4.9%)	0/60 (<4.9%)	0/60 (<4.9%)	0/60 (<4.9%)
Drop one chunk	100 / 95 / 0.0	100 / 0 / 19.0	100 / 95 / 8.1	100 / 95 / 3.2	100 / 95 / 0.5
Reorder two chunks	100 / 100 / 0.0	100 / 0 / 20.0	100 / 100 / 8.9	100 / 100 / 3.8	100 / 100 / 0.5
Duplicate a chunk	100 / 100 / 0.0	100 / 0 / 21.0	100 / 100 / 8.1	100 / 100 / 3.2	100 / 100 / 0.5
Replay a foreign chunk	100 / 100 / 0.0	100 / 100 / 9.7	100 / 100 / 4.5	100 / 100 / 2.0	100 / 100 / 0.5
Replay a whole session	100 / 100 / 0.0	100 / 100 / 0.0	100 / 100 / 0.0	100 / 100 / 0.0	100 / 100 / 0.0
Truncate	100 / 0 / 0.0	100 / 0 / 10.4	100 / 0 / 4.6	100 / 0 / 2.0	100 / 0 / 0.5
Strip all checkpoints	–	–	100 / 100 / 9.0	100 / 100 / 4.0	100 / 100 / 1.0
v2 client (no schedule check)	–	–	0 / 0 / 20.0	0 / 0 / 20.0	0 / 0 / 20.0
Move a checkpoint	–	–	100 / 100 / 9.0	100 / 100 / 4.0	100 / 100 / 1.0
Drop chunk before checkpoint	–	–	100 / 100 / 8.0	100 / 100 / 3.0	100 / 100 / 0.0
Drop chunk after checkpoint	–	–	100 / 100 / 8.0	100 / 100 / 3.0	100 / 100 / 0.0
Reorder across checkpoint	–	–	100 / 100 / 9.0	100 / 100 / 4.0	100 / 100 / 1.0

TABLE VII

Full man-in-the-middle (key substitution) against a live server, pooled over 6 configurations (results were identical in each). Accepted: the client accepted the response; Read: the attacker decrypted the request; Forged: the client accepted an attacker-chosen prediction. Pinned: the client checks the handshake transcript signature under a pinned server identity key (Section IV). Counts out of 1,800 trials per row (300 per configuration).

Client	Attack	Accepted	Read	Forged	Rejected
Unauth.	None	1,800	0	0	–
	Substitute	1,800	1,800	1,800	–
	Subst. + re-sign	1,800	1,800	1,800	–
	Relay + bit flip	0	0	0	AED
Pinned	None	1,800	0	0	–
	Substitute	0	0	0	handshake
	Subst. + re-sign	0	0	0	handshake
	Relay + bit flip	0	0	0	AED

TABLE VIII

Cost of authenticating the handshake (10 connections, 5 repetitions each, run back to back with the unauthenticated configuration).

The identity key uses the configuration’s signature scheme (ML-DSA-65 for Full PQC and Hybrid-KEX-PQ, ECDSA P-256 otherwise). +B: bytes added per handshake; verify: client transcript check; Δ : end-to-end median difference with 95% CI.

Configuration	+B	Verify (μ s)	Δ (ms)
Classical-RSA	71	200	+4.7 [−4.4, +17.2]
Classical-ECDHE	71	86	+0.4 [−2.5, +3.5]
Hybrid	71	86	−0.7 [−2.6, +1.6]
Hybrid-KEX	71	86	+0.2 [−2.0, +2.1]
Full PQC	3,309	46	−1.4 [−3.7, +2.0]
Hybrid-KEX-PQ	3,309	46	+1.7 [−1.0, +4.0]

Hybrid-KEX, and +76.8 ms [+67.3, +89.5] for Classical-RSA, against a 92 ms transaction. All four ML-KEM configurations are equivalent to the X25519 baseline within $\pm 5\%$ at 10 connections (Table XI). This confirms, for inference payloads, what post-quantum TLS studies

TABLE IX

Tampering with single responses: one flipped bit per response, in the AEAD ciphertext or in the signature. Detection time is the rejecting check alone (median).

Config.	Target	n	Detected	Detection (ms)
Classical-RSA	ciphertext	100	100%	0.007
Classical-RSA	signature	100	100%	0.063
Classical-ECDHE	ciphertext	100	100%	0.008
Classical-ECDHE	signature	100	100%	0.083
Hybrid	ciphertext	100	100%	0.008
Hybrid	signature	100	100%	0.083
Hybrid-KEX	ciphertext	100	100%	0.008
Hybrid-KEX	signature	100	100%	0.082
Full PQC	ciphertext	100	100%	0.008
Full PQC	signature	100	100%	0.059
Hybrid-KEX-PQ	ciphertext	100	100%	0.009
Hybrid-KEX-PQ	signature	100	100%	0.058

found for generic traffic [24], [25]: replacing X25519 with ML-KEM-768 adds no practically significant computation. Classical-RSA’s overhead sits in the handshake, where the server generates a fresh RSA-2048 key pair (Fig. 8).

At 1,000 connections, which on a single host is a stress test of client and server contention, the non-RSA protected configurations take 18.9–20.5 s against 9.4 s for the control—but Control-2RT takes 21.3 s, so the doubling comes from the second HTTP request, not the cryptography. Measured against Control-2RT, the protected configurations come out 0.8–2.4 s faster. A negative cryptographic overhead has no physical meaning: it shows that under this contention the subtraction no longer isolates cryptography, so the matched-protocol decomposition is valid only at low concurrency. We therefore draw no equivalence or difference verdicts among the non-RSA configurations at 1,000 connections; their medians are indistinguishable under contention (within -5.4 to $+2.6\%$ of Classical-ECDHE), and only Classical-RSA (30.7 s) is

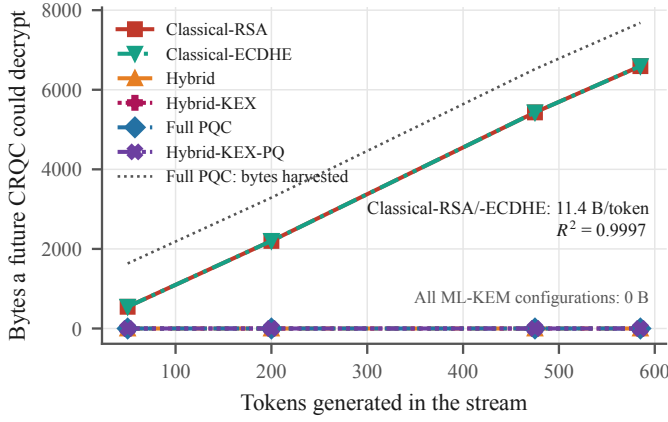


Fig. 6. Streaming HNDL exposure on Llama-3.2-3B. Under classical key exchange (RSA or X25519) the bytes a future CRQC could decrypt grow by 11.4 per generated token; under every ML-KEM configuration they stay at zero although more bytes are harvested.

clearly separated.

Failed requests: At 1,000 connections 1.2–1.6% of protected requests failed, against 0.2% for Control-2RT and none for the control (Table XII). Every failure was a connection reset (httpx ReadError); none was a timeout or an HTTP error, so the excluded requests were not slow requests that timed out. Control-2RT’s failures all occurred on its first request, those of the protected configurations on both requests. Re-counting every failed request at the 30s client timeout changed each median by at most 0.19s and each difference from Classical-ECDHE by at most 1.5 percentage points, so excluding failures does not change the comparisons. Below 1,000 connections at most 0.2% of requests failed.

100 connections: Here the per-repetition medians of the protected configurations are bimodal—each repetition settles near either $\approx 1,000$ ms or $\approx 1,550$ ms (e.g. Classical-ECDHE 1,014–1,701 ms, Full PQC 988–1,604 ms; Hybrid-KEX-PQ stayed in the slower mode in all five, 1,562–1,626 ms)—while both controls are stable (Control-2RT 982–1,003 ms). We attribute the regime switching to queueing with client and server on one CPU and do not rank configurations at this level; only Classical-RSA (+1.39s over Control-2RT) is clearly separated.

Server resources: Classical-RSA consumed 3.5–4.8 cores of server CPU (Table XIII) against about one core or less for every other configuration, because RSA-2048 key generation runs at 16 operations/s (median 62.7 ms over 200 samples) against 98,357/s for ML-KEM-768, a $6,167\times$ gap (Table XVI, Fig. 12). It nevertheless failed only 1.5% of requests at 1,000 connections on this 8-core host. Apart from Classical-RSA, CPU and memory were flat across configurations: at 10 connections every non-RSA configuration, including the control, used 0.92–0.96 cores and 231–241 MB of resident memory.

I. Payload Shape

Across the four profiles the cryptographic overhead is roughly constant in absolute terms (Fig. 9, Table XIV): +4.0–9.4 ms for every non-RSA configuration and +71.6–110.2 ms for Classical-RSA. In relative terms it ranges from +4.5% for the 89 ms RandomForest to up to +240% for millisecond-scale synthetic profiles. The cost of protection is thus an additive term independent of payload size; whether it matters depends on the inference time it is added to. Configurations using X25519 (Classical-ECDHE, Hybrid-KEX) are consistently 1.6–3.5 ms slower than those using ML-KEM alone, matching the primitive costs: server-side handshake work is 217.9 μ s for X25519 key generation plus exchange against 23.3 μ s for ML-KEM key generation plus decapsulation in our implementation.

J. Network Conditions

On emulated paths (Fig. 10) the handshake leg isolates the cost of key-exchange size. Relative to Classical-ECDHE, the ML-KEM configurations add $+4.6 \pm 0.5$ ms (Hybrid), $+4.7 \pm 0.3$ ms (Full PQC), and $+5.7 \pm 1.3$ ms (Hybrid-KEX) on the mobile path, and at most 1.0 ms on the metro and WAN paths. End-to-end, these differences are smaller than the between-repetition variability of server queueing (standard deviation 15–26 ms on WAN and mobile), so they are not resolvable at the application level. Classical-RSA’s handshake is 61–81 ms slower than Classical-ECDHE’s on every path because of key generation, not bytes. Because the proxy does not model TCP slow start or loss, these are lower bounds.

K. Session Reuse

Fig. 11 reports overhead versus the control as one key exchange serves N requests. Classical-RSA’s +101.2% at $N=1$ disappears at $N=10$ ($-1.1 \pm 1.0\%$). For the other configurations the overhead falls from +4.0–7.0% at $N=1$ to between -0.2 and $+2.1\%$ at $N=100$; one intermediate point (Full PQC at $N=10$, $+9.0 \pm 8.4\%$) reflects a single outlying repetition. For deployments that reuse sessions, post-quantum key establishment cost at most about 2% end-to-end in this workload. This was measured on the non-streaming endpoint; our streaming path uses one handshake per response and does not resume sessions (Section X).

VII. Ground-Truth Validation

All 130 tested NIST ACVP known-answer vectors pass (Table XV). Our ML-KEM-768 key generation, encapsulation, and decapsulation, and our ML-DSA-65 key generation (public and secret key) and signing (15 deterministic and 15 hedged vectors, external interface with context strings) reproduce the published outputs byte for byte, and ML-DSA-65 verification returns the expected verdict on every tested vector. For key generation and signing, liboqs drew exactly the 32 random bytes FIPS 204 specifies and no others, so the comparison covers

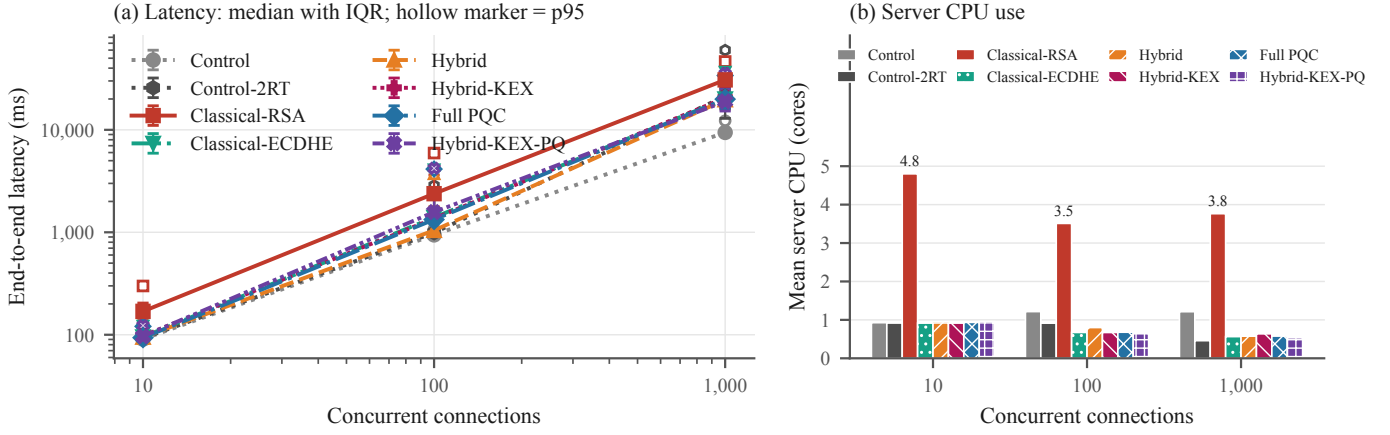


Fig. 7. Concurrency sweep. (a) End-to-end latency (median with interquartile range; hollow markers: p95) on log axes. The non-RSA protected configurations overlap at every level and, at 1,000 connections, match the matched-protocol Control-2RT; Classical-RSA is consistently slowest. (b) Mean server CPU in cores (100% = one core): per-handshake RSA-2048 key generation occupies 3.5–4.8 cores, every other configuration about one core or less.

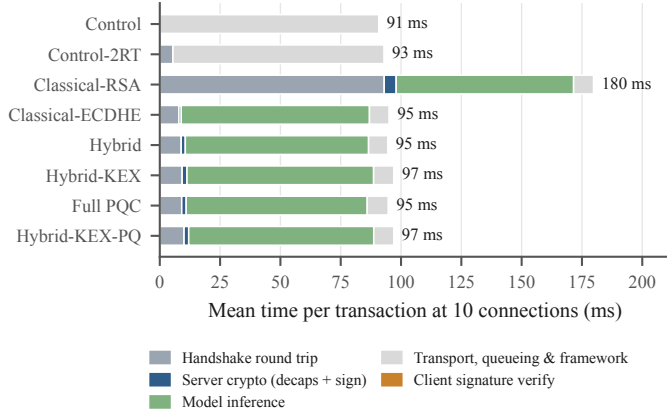


Fig. 8. Mean time per transaction at 10 connections, decomposed. Classical-RSA's extra cost sits in the handshake (server-side RSA-2048 key generation); for the other configurations the handshake is ≈ 8 ms and model inference dominates.

the full computation. This checks the correctness of our bindings on these vectors; it is not ACVP certification, and it covers only the external, pure ML-DSA interface that liboqs implements.

Signature-count and signature-byte predictions of the analytic model matched the harness exactly for the three strategies and for all 18 checkpoint cells (for example 41 ML-DSA-65 signatures, 135,669 B, at $k=1$). For signing time, the model gives bounds rather than a prediction: per-signature cost in a stream lies between the back-to-back and the spaced microbenchmark (Section VI-C). A separate, smaller effect is a one-time cost: ECDSA P-256's first signing call in a fresh process took 7.7–9.3 ms (lazy OpenSSL initialization), which the discarded warm-up generation absorbs.

TABLE X

End-to-end latency (handshake + protected request) by configuration and concurrency, over successful requests after 5% warm-up trimming. “Rep. range”: spread of the five per-repetition medians. Δ : difference in medians (ms) with a 95% two-stage bootstrap CI (repetitions, then requests), versus Control and versus Control-2RT, the matched no-crypto protocol (making Δ_{2RT} the cryptographic overhead alone). δ : Cliff’s delta versus Control. p : two-sided Mann–Whitney U on per-repetition medians (5 vs. 5; smallest attainable 0.008), Holm-corrected within each concurrency level.

Conc.	Configuration	Median (ms)	Rep. range (ms)	p95 (ms)	Fail	Δ vs. Control	Δ_{2RT} (crypto)	δ	p_{Holm}
10	Control	90.7	89.1–92.3	117.0	0.0%	ref.	–	–	–
	Control-2RT	92.4	90.7–94.6	118.8	0.0%	+1.7 [−0.9, +4.0]	ref.	+0.10	0.222
	Classical-RSA	169.2	158.2–185.7	299.3	0.0%	+78.5 [+69.3, +91.3]	+76.8 [+67.3, +89.5]	+0.93	0.056
	Classical-ECDHE	95.2	93.7–96.8	121.6	0.0%	+4.4 [+2.2, +6.6]	+2.7 [+0.6, +5.2]	+0.19	0.056
	Hybrid	94.7	93.3–96.2	117.5	0.0%	+4.0 [+1.9, +6.7]	+2.3 [+0.3, +5.0]	+0.17	0.056
	Hybrid-KEX	97.8	96.7–99.8	123.4	0.0%	+7.1 [+4.5, +9.6]	+5.4 [+3.2, +7.8]	+0.27	0.056
	Full PQC	93.7	93.0–94.2	120.6	0.0%	+3.0 [+1.0, +5.2]	+1.3 [−0.4, +3.7]	+0.14	0.056
	Hybrid-KEX-PQ	96.6	94.1–98.4	122.9	0.0%	+5.9 [+3.2, +8.7]	+4.2 [+1.9, +7.1]	+0.25	0.056
100	Control	942.9	922.8–976.1	1,084.1	0.0%	ref.	–	–	–
	Control-2RT	990.7	982.4–1,003.4	2,854.9	0.0%	+47.8 [+29.5, +67.2]	ref.	+0.24	0.056
	Classical-RSA	2,381.4	2,066.9–2,592.8	5,935.1	0.1%	+1.44 [+1.25, +1.59]s	+1.39 [+1.20, +1.53]s	+0.86	0.056
	Classical-ECDHE	1,396.2	1,014.0–1,701.4	4,081.1	0.1%	+453.3 [+121.4, +704.6]	+405.5 [+68.4, +654.8]	+0.50	0.056
	Hybrid	1,043.7	983.5–1,590.4	3,719.4	0.0%	+100.8 [+40.4, +416.4]	+53.0 [−4.9, +373.2]	+0.39	0.056
	Hybrid-KEX	1,361.7	1,115.1–1,638.5	4,179.9	0.2%	+418.9 [+196.7, +665.8]	+371.0 [+148.7, +608.6]	+0.47	0.056
	Full PQC	1,331.9	987.8–1,604.3	4,139.3	0.1%	+389.0 [+86.9, +646.4]	+341.2 [+38.8, +594.2]	+0.44	0.056
	Hybrid-KEX-PQ	1,576.4	1,561.8–1,626.0	4,209.7	0.2%	+633.5 [+592.1, +685.1]	+585.7 [+545.6, +634.7]	+0.53	0.056
1,000	Control	9,428.0	9,402.4–9,586.0	12.27s	0.0%	ref.	–	–	–
	Control-2RT	21.31s	18.92s–23.89s	59.93s	0.2%	+11.88 [+10.46, +13.74]s	ref.	+0.77	0.056
	Classical-RSA	30.73s	29.94s–31.23s	46.42s	1.5%	+21.30 [+20.86, +21.63]s	+9.42 [+7.58, +10.87]s	+0.98	0.056
	Classical-ECDHE	20.00s	19.57s–20.37s	36.62s	1.5%	+10.57 [+10.24, +10.80]s	−1.31 [−3.26, +0.18]s	+0.92	0.056
	Hybrid	19.66s	19.09s–20.10s	34.23s	1.5%	+10.24 [+9.85, +10.55]s	−1.65 [−3.55, −0.12]s	+0.93	0.056
	Hybrid-KEX	20.52s	19.41s–22.53s	35.41s	1.2%	+11.09 [+10.28, +12.44]s	−0.79 [−2.88, +1.18]s	+0.95	0.056
	Full PQC	19.92s	19.68s–20.23s	33.88s	1.6%	+10.49 [+10.24, +10.57]s	−1.39 [−3.21, −0.01]s	+0.93	0.056
	Hybrid-KEX-PQ	18.93s	18.24s–20.33s	35.31s	1.3%	+9.50 [+8.94, +10.50]s	−2.38 [−4.41, −0.58]s	+0.89	0.056

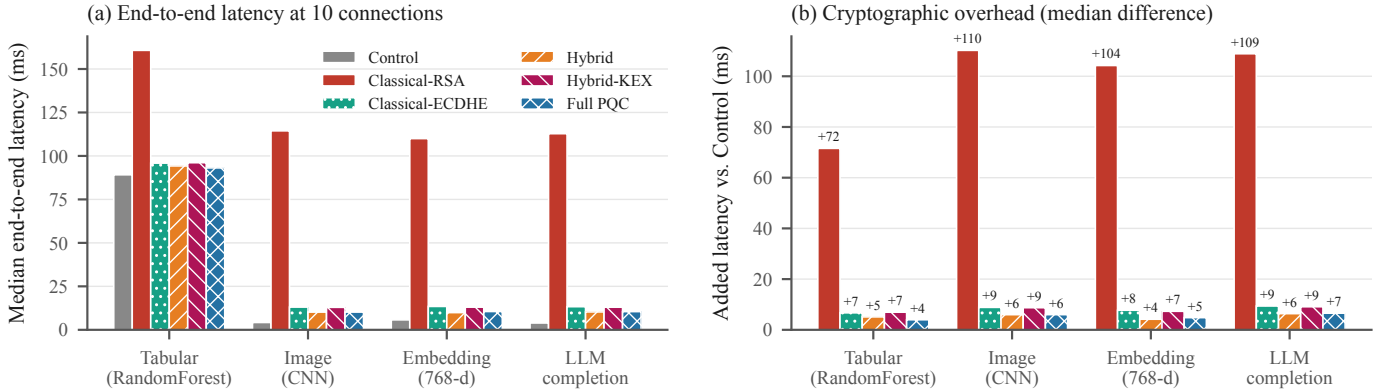


Fig. 9. Payload-shape sensitivity at 10 connections. (a) Median end-to-end latency per profile. (b) Added latency versus the control: roughly constant in milliseconds across profiles, so the cryptographic cost is additive and independent of payload size.

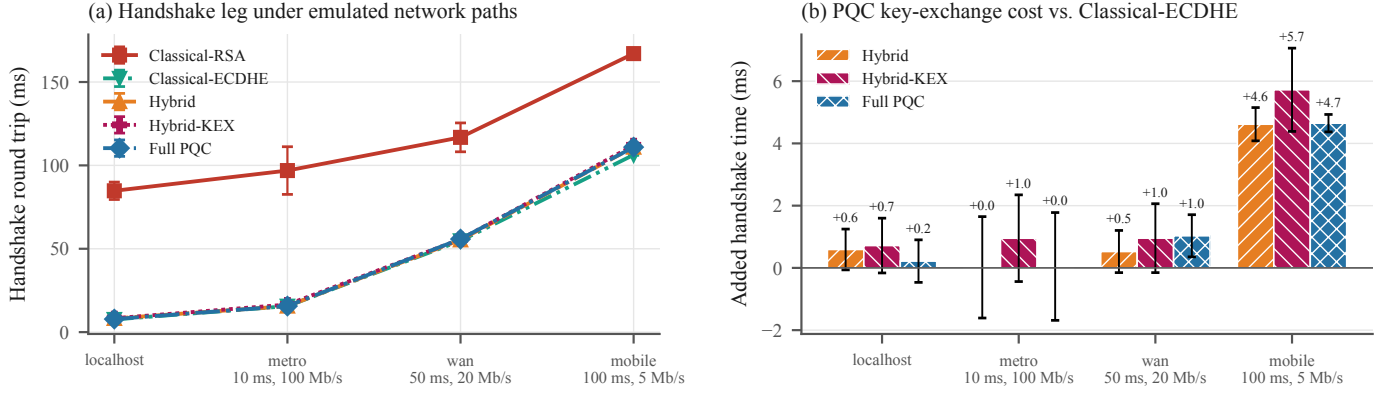


Fig. 10. Emulated network paths (10 connections; mean of five per-repetition medians, error bars one standard deviation). (a) Handshake round trip. (b) Added handshake time of the ML-KEM configurations relative to Classical-ECDHE in the byte-level emulation: about 5 ms on the 5Mbit/s mobile path, at most 1 ms elsewhere (a lower bound).

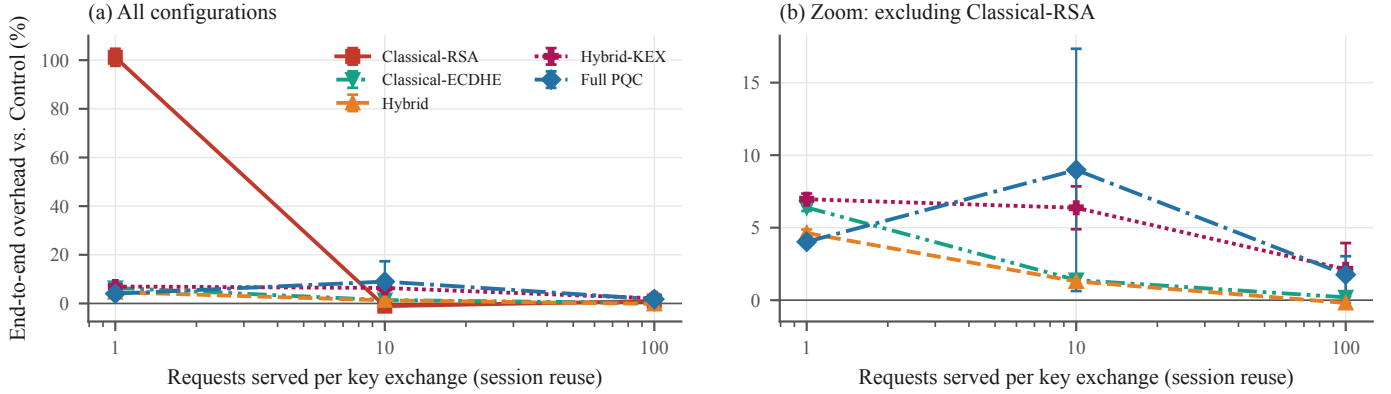


Fig. 11. Session reuse at 10 connections (mean $\pm$ one standard deviation over five repetitions). (a) All configurations: Classical-RSA's overhead vanishes once a key exchange serves ten requests. (b) Zoom without Classical-RSA: post-quantum overhead falls to 0–2% at 100 requests per key exchange.

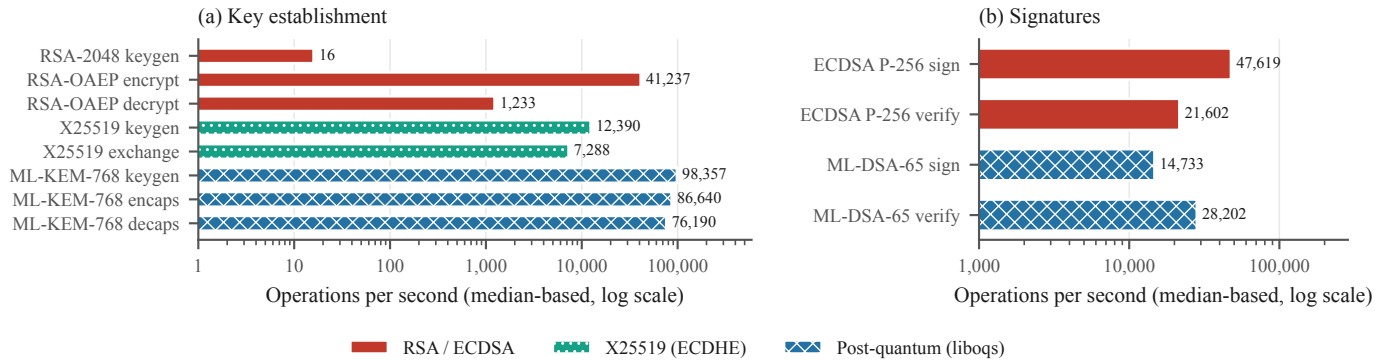


Fig. 12. Primitive throughput (single process, warm loop, median-based). (a) Key establishment, log scale: RSA-2048 key generation is $6,167\times$ slower than ML-KEM-768 key generation, and ML-KEM-768 is also cheaper than X25519. (b) Signatures: ML-DSA-65 signing is $3.2\times$ slower than ECDSA P-256, while its verification is $1.31\times$ faster.

TABLE XI

Equivalence of ML-KEM-based key establishment to the X25519 baseline (Classical-ECDHE): median end-to-end difference with its 90% two-stage bootstrap CI, as a percentage of the Classical-ECDHE median. “Equiv.”: the CI lies within $\pm 5\%$ (two one-sided tests at $\alpha = 0.05$). A verdict is given only at 10 connections. At 100 connections the protected configurations are bimodal, and at 1,000 client–server contention on the shared host dominates (the protected configurations are faster than the no-crypto Control-2RT), so differences there are shown for completeness and are not interpreted as equivalence or difference.

Conc.	Configuration	Δ (%) [90% CI]	Equiv.
10	Hybrid	-0.5 [-2.2, +1.8]	yes
	Hybrid-KEX	+2.8 [+0.8, +5.0]	yes
	Full PQC	-1.5 [-3.1, +0.2]	yes
	Hybrid-KEX-PQ	+1.6 [-0.3, +3.9]	yes
100	Hybrid	-25.2 [-43.1, +1.7]	n/i
	Hybrid-KEX	-2.5 [-26.9, +27.3]	n/i
	Full PQC	-4.6 [-35.6, +29.4]	n/i
	Hybrid-KEX-PQ	+12.9 [-4.0, +36.3]	n/i
1,000	Hybrid	-1.7 [-3.7, +0.4]	n/i
	Hybrid-KEX	+2.6 [-1.2, +8.2]	n/i
	Full PQC	-0.4 [-1.9, +1.0]	n/i
	Hybrid-KEX-PQ	-5.4 [-8.5, -0.8]	n/i

TABLE XII

Failed requests and their effect on the comparisons. All failures were connection resets (httpx ReadError): no timeouts and no HTTP errors. “At handshake”: the first request of the transaction failed. Sensitivity: every failed request re-counted at the 30 s client timeout (slower than any successful request) and the statistics recomputed. Δ : difference in medians versus Classical-ECDHE, % of its median, with 95% CI. These levels are dominated by single-host contention, so the table tests only whether excluding failures changes the comparisons; it draws no equivalence verdicts.

Conc.	Configuration	Failed	Failed at		Median (s)		Δ vs. Classical-ECDHE (%)	
			handshake	request	succ. only	fail. at 30 s	successful only	failures at 30 s
1,000	Control	0.0%	–	0	9.43	9.43	–	–
	Control-2RT	0.2%	95	0	21.31	21.34	–	–
	Classical-RSA	1.5%	345	362	30.73	30.61	+53.6 [+50.7, +55.9]	+52.1 [+49.6, +54.6]
	Classical-ECDHE	1.5%	306	389	20.00	20.12	ref.	ref.
	Hybrid	1.5%	292	412	19.66	19.85	-1.7 [-4.1, +0.9]	-1.3 [-3.5, +0.7]
	Hybrid-KEX	1.2%	276	317	20.52	20.63	+2.6 [-1.7, +9.8]	+2.5 [-1.8, +8.9]
	Full PQC	1.6%	330	430	19.92	19.93	-0.4 [-2.1, +1.2]	-0.9 [-1.9, +1.0]
	Hybrid-KEX-PQ	1.3%	274	334	18.93	19.07	-5.4 [-8.7, +0.3]	-5.2 [-8.8, +0.5]

TABLE XIII

Server-process resource use, sampled every 0.25 s during each cell and averaged over repetitions; RSS is mean resident memory (macOS, Apple M3, 8 logical CPUs; 100% = one core).

Conc.	Config.	CPU mean (%)	CPU max (%)	RSS (MB)	Req/s
10	Control	93	126	231	107.0
	Control-2RT	92	126	231	104.4
	Classical-RSA	480	699	238	52.4
	Classical-ECDHE	92	125	238	101.6
	Hybrid	92	126	239	102.1
	Hybrid-KEX	92	125	241	99.4
	Full PQC	94	127	241	102.1
	Hybrid-KEX-PQ	96	126	240	98.9
100	Control	122	128	241	104.8
	Control-2RT	92	131	238	78.8
	Classical-RSA	351	792	247	35.8
	Classical-ECDHE	67	134	245	57.8
	Hybrid	80	130	247	68.1
	Hybrid-KEX	67	132	249	56.7
	Full PQC	68	133	250	58.3
	Hybrid-KEX-PQ	64	133	248	53.0
1,000	Control	121	129	269	102.5
	Control-2RT	46	138	214	38.4
	Classical-RSA	376	800	283	32.9
	Classical-ECDHE	56	137	280	46.1
	Hybrid	58	139	286	47.7
	Hybrid-KEX	64	142	294	45.8
	Full PQC	58	139	290	47.7
	Hybrid-KEX-PQ	53	141	293	48.2

TABLE XIV

Payload-shape sensitivity at 10 connections (5 repetitions per cell). Request/response sizes are plaintext medians; each cell gives median end-to-end latency (ms) and the added latency versus Control, with Mann–Whitney U significance (* $p < 0.05$).

Profile	Req. (B)	Resp. (B)	Control	Classical-RSA	Classical-ECDHE	Hybrid	Hybrid-KEX	Full PQC
tabular_small	348	90	89.2	160.7 (+71.6*)	95.9 (+6.7*)	94.3 (+5.1*)	96.1 (+7.0*)	93.1 (+4.0*)
image_cnn	4,138	245	4.2	114.4 (+110.2*)	13.1 (+8.9*)	10.2 (+6.0*)	12.9 (+8.7*)	10.2 (+6.0*)
embedding	202	8,017	5.7	110.0 (+104.2*)	13.5 (+7.7*)	9.9 (+4.2*)	13.0 (+7.3*)	10.5 (+4.8*)
llm_completion	157	4,048	3.9	112.8 (+108.9*)	13.3 (+9.4*)	10.3 (+6.4*)	13.0 (+9.1*)	10.5 (+6.6*)

TABLE XV

NIST ACVP known-answer-test results (130/130)

Test	Result	Basis
ML-KEM-768 keygen (derand., Alg. 16)	25/25	byte-exact
ML-KEM-768 encapsulation (Alg. 17)	25/25	byte-exact
ML-KEM-768 decapsulation (incl. 5 implicit-rej.)	10/10	byte-exact
ML-DSA-65 keygen (seed ξ , Alg. 1)	25/25	byte-exact
ML-DSA-65 signing, deterministic (Alg. 2)	15/15	byte-exact
ML-DSA-65 signing, hedged (Alg. 2)	15/15	byte-exact
ML-DSA-65 signature verification (Alg. 3)	15/15	correct verdict

TABLE XVII

Software and hardware environment, captured automatically at run time.

Component	Version
Host	Apple M3, 8 cores, 16 GB
OS	macOS 26.6.2
Python	3.13.5
FastAPI / Uvicorn / httpx	0.141.1 / 0.52.4 / 0.28.1
cryptography / OpenSSL	50.0.0 / 4.0.1
liboqs (commit)	8979276ad1eb
NumPy / SciPy / pandas	2.5.2 / 1.18.1 / 3.0.5
scikit-learn	1.9.0
llama-cpp-python (Metal)	0.3.35
LLM file	Llama-3.2-3B-Instruct-Q4_K_M.gguf
LLM SHA-256 (prefix)	6c1a2b4116103267
Classifier SHA-256 (prefix)	68b46ef7e1efce32

TABLE XVI

Primitive-level micro-benchmark (single process, warm loop). n = timed iterations.

Algorithm	Op.	n	Median (μ s)	p99 (μ s)	ops/s
ML-KEM-768	KeyGen	2000	10.2	12.5	98,357
ML-KEM-768	Encaps	2000	11.5	14.7	86,640
ML-KEM-768	Decaps	2000	13.1	15.0	76,190
X25519	KeyGen	2000	80.7	93.5	12,390
X25519	Exchange	2000	137.2	150.1	7,288
RSA-2048	KeyGen	200	62,698.0	257,378.1	16
RSA-2048 OAEP	Encrypt	2000	24.3	27.3	41,237
RSA-2048 OAEP	Decrypt	2000	811.3	1,730.7	1,233
ML-DSA-65	KeyGen	2000	38.0	42.3	26,316
ML-DSA-65	Sign	2000	67.9	246.1	14,733
ML-DSA-65	Verify	2000	35.5	40.8	28,202
ECDSA P-256	KeyGen	2000	15.5	18.5	64,687
ECDSA P-256	Sign	2000	21.0	23.7	47,619
ECDSA P-256	Verify	2000	46.3	54.2	21,602

VIII. Discussion and Decision Summary

An earlier version of this paper combined an ordinal security score with a latency overhead into one weighted number. We no longer do so: the two have no common unit, the security score was assigned rather than derived, and the overhead it used (versus the single-request control) was dominated by the extra HTTP round trip that Control-2RT shows is not cryptographic. Table XVIII instead states three yes/no security properties of each construction and its measured costs, and asks only which configurations no other configuration beats on every axis at once. Whatever weights a reader attaches to the axes, the preferred configuration lies in that non-dominated set.

The result is sharper than a ranking. Classical-RSA is dominated by Classical-ECDHE in both workloads: it is slower and no more secure. No other configuration dominates another. Latency, compared at 10 connections where it can be interpreted, does not separate them—every non-RSA configuration is within the equivalence margin of the others or within a few milliseconds of them—so every step up in security is paid for in bytes, not time: ML-KEM adds about 1 kB to the handshake, ML-DSA-65 about 3.2 kB per signature, and with per-chunk streaming about 133 kB per 200-token response. Which non-dominated configuration to choose therefore depends on which security properties a deployment needs and on how many signatures it sends, not on latency. This is also why the earlier composite score favoured Full PQC: it weighted a security score that Full PQC maximizes against a latency term that barely varies.

Hypotheses: H1 (bounded post-quantum overhead) is supported at 10 connections, where all four ML-KEM configurations are equivalent to Classical-ECDHE within $\pm 5\%$ and their cryptographic overhead over the matched protocol is 1.3–5.4 ms. At 100 and 1,000 connections the single host prevents a conclusion (bimodality and contention, respectively). H2 (Hybrid captures Full PQC’s HNDL benefit at lower cost) is supported on HNDL, where both have zero decryptable bytes under the modeled attack; on cost it holds only for signature bytes, which matter when many signatures are sent. H3 (categorically different decryptability) is demonstrated: classical exposure grows linearly with streamed tokens, ML-KEM-based exposure stays at zero. H4 (ML-DSA-65 does not slow responses) needs restating. ML-DSA-65 verification is faster than ECDSA P-256 (28,202 versus 21,602 per second), but verification is the client’s cost; the server’s cost is signing, where ML-DSA-65 is $3.2\times$ slower (67.9 versus 21.0 μ s median). That difference, about 47 μ s per response back to back and about 180 μ s per signature in the spaced pattern of a stream, is under 0.2% of a 92 ms transaction and is not resolvable end to end (Table X). H4 holds on that basis, not on verification speed.

IX. Recommendations

These follow from our measurements under our threat model; each is a design choice for an application-layer protocol, not a statement about TLS stacks.

Authenticate the handshake: Post-quantum key establishment does not stop an active adversary who substitutes keys; only authentication of the server’s keys does (Section VI-E). Sign the handshake transcript with an identity key the client can verify (in TLS, a certificate and CertificateVerify), using ML-DSA-65 where quantum-resistant authentication is required. In our measurements this cost one signature per handshake and no resolvable latency.

Key establishment: Within the modeled threat, replacing X25519 with ML-KEM-768 removes the HNDL key-recovery exposure at no practically significant computational cost (equivalent within $\pm 5\%$ at 10 connections), about 5 ms of handshake time on a slow emulated link (a lower bound), and 0–2% once sessions are reused. Where hedging against an ML-KEM weakness matters, X25519MLKEM768 cost about 3 ms more than ML-KEM alone at 10 connections. Per-handshake RSA key generation should be avoided: it occupied 4–5 CPU cores at modest load.

Signatures: For single responses, choose the signature by the required security property, not by latency: ML-DSA-65 signing costs about 47 μ s more per response than ECDSA P-256, which was not measurable end to end, while its 3,309 B signature adds bytes to every response.

Streaming: Use hash-chained signing with checkpoints, bind role, session, and position into every signed message, end every stream with a signed length record, and have the client enforce the checkpoint schedule it requested (Section VI-D). Choose k from the exposure the application can tolerate: $k=\infty$ (one signature; the response is unverified until it ends) only where unverified text is harmless, small k where a user must not act on it. With ML-DSA-65, $k=5$ bounds exposure to 25 tokens for 29,781 B per 200-token response, against 135,669 B for per-chunk signing. Remember that detection is not prevention: a client should mark text as unverified until a signature covers it, and withdraw it if verification fails. Buffer-and-sign is justified only when chunk-count and timing metadata must be hidden [29], at a $40\times$ time-to-first-token penalty.

TABLE XVIII

Decision summary without weights. Security columns are properties of the construction ($\checkmark$ = holds): key establishment resists a CRQC (KEX-Q); key establishment survives a break of ML-KEM (KEX-H); signatures resist a CRQC (Sig-Q). Δ_{2RT} : cryptographic overhead versus Control-2RT (median, 95% CI). Bytes: key-establishment blob + one signature per transaction (API), and key-establishment blob + signatures on a 200-token per-chunk stream. A configuration is dominated if another is at least as secure on every property, no worse on latency at 10 connections (90% CI of the difference below +5%), no larger in the workload’s bytes, and strictly better somewhere; the last two columns name the dominating configurations (codes as in Table II), or “–” if none. The 100- and 1,000-connection levels are not used: the first is bimodal and the second dominated by single-host contention (Section VI-H).

Configuration	Security			Δ_{2RT} @10 (ms)	Bytes		Dominated by	
	KEX-Q	KEX-H	Sig-Q		API	Per-chunk	API	Per-chunk
A: Classical-RSA	–	$\checkmark$	–	+76.8 [+67.2, +88.2]	327	3,239	A'	A'
A': Classical-ECDHE	–	$\checkmark$	–	+2.7 [+0.7, +5.4]	102	3,014	–	–
B: Hybrid	$\checkmark$	–	–	+2.3 [+0.5, +5.2]	1,159	4,066	–	–
B': Hybrid-KEX	$\checkmark$	$\checkmark$	–	+5.4 [+3.1, +7.9]	1,190	4,102	–	–
C: Full PQC	$\checkmark$	–	$\checkmark$	+1.3 [–0.3, +3.7]	4,397	140,066	–	–
C': Hybrid-KEX-PQ	$\checkmark$	$\checkmark$	$\checkmark$	+4.2 [+1.9, +7.0]	4,429	140,098	–	–

TABLE XIX
Recommendation summary

Scenario	Recommendation
Any protected API	Authenticate the handshake (transcript signature under a trusted identity key)
Key establishment	ML-KEM-768, or X25519MLKEM768 to hedge against an ML-KEM break
Single responses	ECDSA P-256 or ML-DSA-65 by required security; latency does not decide
Streaming, display-only	Hash-chain, large k ; mark text unverified until covered
Streaming, actionable output	Hash-chain with small k (e.g. $k = 5$: 25-token exposure)
Any streaming	Bind session context; signed end record; enforce the checkpoint schedule
Quantum-resistant authenticity	ML-DSA-65 (Full PQC; or Hybrid-KEX-PQ, less measured)
Metadata-sensitive streaming	Buffer-and-sign
Legacy RSA key transport	Migrate away

X. Limitations and Threats to Validity

Internal validity:

- Single host, shared CPU. All measurements come from one Apple M3 laptop on which the load generator and the server share the CPU. This matters most at high concurrency: at 1,000 connections both the protected configurations and Control-2RT are dominated by contention, and at 100 connections the protected configurations switch between two latency regimes across repetitions while both controls do not. At 1,000 connections the protected configurations even come out faster than the no-crypto Control-2RT, so the matched-protocol decomposition holds only at low concurrency. We therefore draw equivalence and ranking conclusions only at 10 connections and treat 100 and 1,000 connections as single-host stress

tests. A separate load-generator host would remove this confound; we did not have one.

- Failed requests. At 1,000 connections 0.2–1.6% of requests failed, all as connection resets. Excluding them could bias latency toward configurations that fail more; the sensitivity analysis in Section VI-H counts each failure at the client timeout instead.
- Processor power state. Per-signature time inside a stream depends on how far the core has clocked down between chunks (Section VI-C); on a busy server it would be closer to the back-to-back microbenchmark. We report wall-clock and CPU time and give both bounds, and we compare signature bytes, which are exact, rather than signing times across strategies.
- Thermal drift. Generation slowed from ≈ 40 to ≈ 33 tokens/s during streaming sweeps on the fanless host. Cell order was shuffled within each configuration, so strategy comparisons are unbiased, but streaming times are not compared across configurations.

Construct validity:

- Application-layer emulation, not TLS. PQ-Shield reproduces the cryptographic work of a protected transaction with a purpose-built protocol: no TLS framing, record layer, PKI, or TLS resumption (PSK, 0-RTT). Its default fresh key exchange per transaction is a worst case; Section V-G measures the amortized case. The default handshake is unauthenticated, so the default configurations are not resistant to key substitution (Section VI-E); the authenticated variant pins one identity key and does not model certificate chains.
- Byte-level network emulation. The proxy models propagation delay and bandwidth but not slow start, the initial congestion window, loss, or fragmentation, so network costs are lower bounds; packet-level studies [24], [25] remain the reference.
- HNDL measures bytes, not information. D counts ciphertext bytes that become decryptable under the modeled attack; it does not measure what an adver-

sary learns from them, and the per-token rate depends on chunk size and tokenizer. Response length leaks without a CRQC [29].

- Attack trials sample positions. Campaign trials replay attacks on captured streams at random positions; they cover the attack space but are not independent network events, and they test the attacks listed, not every possible adversary. With the stream as the unit, each entry rests on 60 streams, so the detection bounds (94.0% of streams at 95% confidence) are much wider than the trial counts suggest.
- Classical-RSA is a worst case. Per-handshake RSA-2048 key generation does not reflect production practice; Classical-ECDHE is the realistic classical baseline.

External validity:

- One host class and one model. Results come from Apple silicon with OpenSSL and liboqs; server-class x86 CPUs, other libraries, and GPU-served models may differ in absolute terms. Streaming uses one quantized 3B-parameter model.
- Workloads. The concurrency experiments use a RandomForest classifier; the `image_cnn` profile is an untrained fixed-weight network and the embedding and `llm_completion` profiles are synthetic. They probe payload shape and size, not model behaviour, so the absolute overheads should be read against the inference time of the target workload.
- Coverage of C' . Hybrid-KEX-PQ was added in revision. It is absent from the payload, network, and session-reuse sweeps, and its concurrency data come from a separate run about seven hours after the main sweep (same host, same day; Hybrid-KEX's likewise come from a supplementary run). The configuration with the strongest security properties is therefore the least measured; where we recommend it, the recommendation rests on its construction and on the concurrency, streaming, checkpoint, threat, and authentication experiments in which it was measured.
- No session resumption on the streaming path. Each stream has its own handshake: the session context in every signed message is derived from the handshake identifier, one stream per handshake. The session-reuse result (Section VI-K) was measured on the non-streaming endpoint and does not carry over: a streaming deployment as designed pays one key exchange per response. With a ≈ 8 ms handshake leg against 5–6 s of generation this is small, but it also means no amortization and one recorded handshake per response for an HNDL adversary. Resuming a session across streams would require a per-stream context (for example, a stream counter bound into *ctx*), which we neither implemented nor measured.
- Side channels. We did not evaluate constant-time behaviour or cache-timing leakage of the bindings or

the streaming verification path.

Statistical conclusion validity: Five repetitions per cell bound the precision of between-run comparisons: with repetition medians as the unit, the smallest attainable p is 0.008, and equivalence can be shown only for differences well inside the $\pm 5\%$ margin.

XI. Conclusion and Future Work

For a streamed LLM response, the post-quantum cost lies in the signature schedule, not the key exchange. Per-chunk ML-DSA-65 signing adds 135,669 B to a 200-token response; one terminal signature adds 3,309 B but leaves the whole response unverified until it ends. A hash chain checkpointed every k chunks spans that range with signature bytes that follow an analytic model exactly. Binding role, session, and position into every signed message, ending each stream with a signed length record, and enforcing the requested checkpoint schedule made every one of the attacks we tested detectable, including truncation and checkpoint stripping. Detection, however, arrives only when a signature does, and k sets how much unverified text a user has seen by then. The answers to the practical questions follow; each holds under our threat model and on our single host.

Strongest security: Hybrid-KEX-PQ (X25519MLKEM768 with ML-DSA-65) is the only evaluated configuration with all three properties we track: key establishment that resists a CRQC, key establishment that survives an ML-KEM break, and quantum-resistant signatures. Full PQC lacks only the hedge. Neither resists key substitution unless the handshake is authenticated.

Best performance: There is no single fastest configuration. At 10 connections every ML-KEM configuration is equivalent to X25519 within $\pm 5\%$, with cryptographic overheads of 1.3–5.4 ms per 92 ms transaction; only Classical-RSA with per-transaction key generation is clearly slower. When many signatures are sent, the ECDSA-signed configurations are cheapest in bytes, by about $47\times$ per signature.

Best balance: Classical-RSA is dominated; among the rest, no configuration is better on every axis, and moving up in security costs bytes rather than latency. For non-streaming APIs and for streaming with few signatures, Full PQC or Hybrid-KEX-PQ gives the strongest security for a few kilobytes per response. When every chunk must be signed or checkpoints are frequent, Hybrid or Hybrid-KEX keeps signatures at ≈ 2.9 kB instead of 136 kB per 200 tokens, at the price of classical signatures.

What the answer depends on: Streaming: the number of signatures, through k , sets the post-quantum cost and the exposure. Session reuse: one key exchange per 100 requests reduced every non-RSA configuration's overhead to 0–2% on the non-streaming endpoint; our streaming path pays one key exchange per response. Threat model: HNDL confidentiality needs ML-KEM; active attacks need

an authenticated handshake; quantum-resistant authenticity needs ML-DSA-65; hedging against an ML-KEM break needs the X25519MLKEM768 hybrid. Network: slow links add a few milliseconds of handshake time for ML-KEM, more under packet loss than our byte-level emulation shows.

Future work includes a separate load-generator host and server-class hardware, packet-level network emulation, a TLS-native implementation with certificate chains and resumption, mid-stream rekeying to bound HNDL exposure per session, larger models and real request traces, other tokenizers for the HNDL rate, and constant-time and side-channel analysis of the streaming verification path.

Acknowledgment

The authors thank the faculty and technical staff of the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, for the resources and infrastructure used in this study, and the open-source cryptographic and machine-learning communities, whose tools and standards supported its implementation, validation, and evaluation.

References

- [1] M. Abbasi, F. Cardoso, P. Váz, J. Silva, and P. Martins, “A practical performance benchmark of post-quantum cryptography across heterogeneous computing environments,” *Cryptography*, vol. 9, no. 2, art. 32, 2025, doi: 10.3390/cryptography9020032.
- [2] E. D. Demir, B. Bilgin, and M. C. Onbasli, “Performance analysis and industry deployment of post-quantum cryptography algorithms,” arXiv:2503.12952, 2025.
- [3] O. Egbuagha and E. Ikwunna, “Post-quantum cryptography in practice: A literature review of protocol-level transitions and readiness,” *IACR Cryptology ePrint Archive*, Paper 2025/1668, 2025.
- [4] P. Pote and R. Bansode, “Performance evaluation of post-quantum cryptography: A comprehensive framework for experimental analysis,” *J. Inf. Syst. Eng. Manage.*, vol. 10, no. 9s, pp. 548–556, 2025, doi: 10.52783/jisem.v10i9s.1253.
- [5] A. Shaw, “quantum-safe: Bridging the post-quantum production gap with a hybrid-by-default Python cryptography library,” arXiv:2605.17061, 2026.
- [6] J. A. Montenegro, R. Rios, and J. Lopez-Cerezo, “A performance evaluation framework for post-quantum TLS,” *Future Gener. Comput. Syst.*, vol. 175, art. 108062, 2026, doi: 10.1016/j.future.2025.108062.
- [7] G. Chhetri, S. Somvanshi, P. Hebli, S. Brotee, and S. Das, “Post-quantum cryptography and quantum-safe security: A comprehensive survey,” arXiv:2510.10436, 2025.
- [8] R. A. Faval, R. Moreira, and F. de Oliveira Silva, “Empowering mobile networks security resilience by using post-quantum cryptography,” arXiv:2603.28626, 2026.
- [9] A. Kudaloor and A. Aijaz, “Toward quantum-safe 6G: Experimental evaluation of post-quantum cryptography techniques,” *IEEE Commun. Standards Mag.*, accepted, 2026, arXiv:2605.06881.
- [10] L. Song, R. Shokri, and P. Mittal, “Privacy risks of securing machine learning models against adversarial examples,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2019, pp. 241–257.
- [11] L. N. Tidjon and F. Khomh, “Threat assessment in machine learning based systems,” arXiv:2207.00091, 2022.
- [12] N. Ahmed, L. Zhang, and A. Gangopadhyay, “A survey of post-quantum cryptography support in cryptographic libraries,” in *Proc. IEEE Int. Conf. Quantum Comput. Eng. (QCE)*, 2025, arXiv:2508.16078.
- [13] J. Blanco-Romero, F. Almenares Mendoza, C. García Rubio, C. Campo, and D. Díaz Sánchez, “On the practical feasibility of harvest-now, decrypt-later attacks,” arXiv:2603.01091, 2026.
- [14] D. Gómez-Cambronero, D. Munteanu, and A. I. González-Tablas, “Layered performance analysis of TLS 1.3 handshakes: Classical, hybrid, and pure post-quantum key exchange,” arXiv:2603.11006, 2026.
- [15] A. G. Schemitt, H. F. da Silva, R. C. Lunardi, D. Kreutz, R. B. Mansilha, and A. F. Zorzo, “Assessing the impact of post-quantum digital signature algorithms on blockchains,” in *Proc. IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, 2025, arXiv:2510.09271.
- [16] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” *FIPS PUB 203*, Aug. 2024, doi: 10.6028/NIST.FIPS.203.
- [17] National Institute of Standards and Technology, “Module-lattice-based digital signature standard,” *FIPS PUB 204*, Aug. 2024, doi: 10.6028/NIST.FIPS.204.
- [18] National Institute of Standards and Technology, “Automated Cryptographic Validation Protocol (ACVP) server and test vectors,” 2024. [Online]. Available: <https://github.com/usnistgov/ACVP-Server>
- [19] E. Rescorla, “The Transport Layer Security (TLS) protocol version 1.3,” *IETF RFC 8446*, Aug. 2018.
- [20] K. Kwiatkowski, P. Kampanakis, B. Westerbaan, and D. Stebila, “Post-quantum hybrid ECDHE-MLKEM key agreement for TLSv1.3,” *IETF Internet-Draft draft-ietf-tls-ecdhe-mlkem-05*, May 2026.
- [21] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997.
- [22] M. Mosca, “Cybersecurity in an era with quantum computers: Will we be ready?” *IEEE Secur. Privacy*, vol. 16, no. 5, pp. 38–41, 2018.
- [23] D. Stebila and M. Mosca, “Post-quantum key exchange for the Internet and the Open Quantum Safe project,” in *Selected Areas in Cryptography – SAC 2016*, ser. LNCS, vol. 10532. Springer, 2017, pp. 14–37.
- [24] C. Paquin, D. Stebila, and G. Tamvada, “Benchmarking post-quantum cryptography in TLS,” in *Post-Quantum Cryptography – PQCrypto 2020*, ser. LNCS, vol. 12100. Springer, 2020, pp. 72–91.
- [25] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, “Post-quantum authentication in TLS 1.3: A performance study,” in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2020.
- [26] R. Gennaro and P. Rohatgi, “How to sign digital streams,” in *Advances in Cryptology – CRYPTO ’97*, ser. LNCS, vol. 1294. Springer, 1997, pp. 180–197.
- [27] C. K. Wong and S. S. Lam, “Digital signatures for flows and multicasts,” *IEEE/ACM Trans. Netw.*, vol. 7, no. 4, pp. 502–513, 1999.
- [28] A. Perrig, R. Canetti, J. D. Tygar, and D. Song, “Efficient authentication and signing of multicast streams over lossy channels,” in *Proc. IEEE Symp. Secur. Privacy*, 2000, pp. 56–73.
- [29] R. Weiss, D. Ayzenshteyn, G. Amit, and Y. Mirsky, “What was your prompt? A remote keylogging attack on AI assistants,” in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 3367–3384.